

A E G I S

From Scattered Telemetry to a Story You Can Trust

Deterministic detection, attack-chain reconstruction and
evidence-grounded AI investigation in a self-hosted security
platform

Raunit Thakur

September 2026

Project thesis · Version 1.0

Source code: github.com/raunitgrey7/aegis

Live demonstration: aegis-ochre-eight.vercel.app

Abstract

Security operations teams do not struggle to generate alerts; they struggle to understand them. A modern intrusion leaves a thin trail of individually unremarkable events — a login at an odd hour, a scripted process launch, an outbound connection, a compressed archive — scattered across authentication, endpoint, network and DNS telemetry, and buried under millions of legitimate records. Commercial tooling answers this with volume: more detections, more dashboards, more disconnected alerts for a human analyst to reconcile. The consequential work, deciding whether six weak signals are one strong attack, is left to whoever is on shift.

This thesis presents Aegis, a self-hosted platform that ingests raw security telemetry, detects malicious behaviour deterministically, correlates the results into incidents, reconstructs each incident as a phase-annotated attack graph mapped to MITRE ATT&CK, scores its risk with a fully explainable formula, and only then hands the assembled evidence to a locally hosted language model — not to decide anything, but to explain what the deterministic engines already found. Every sentence the model produces is validated against the incident's real evidence identifiers before an analyst sees it; a fabricated citation causes the narrative to be discarded in favour of a deterministic report that is always produced. The design principle throughout is that *AI is not the detector*.

The platform comprises a canonical event schema covering twenty event types; a YAML rule language with stateless, threshold and sequence semantics carrying 58 shipped detections; five families of statistical baselines built on robust statistics; local threat-intelligence matching over public feeds; a union-find correlation engine with an explicit incident-admission policy; a security knowledge graph; and a hardened API with role-based access control, a hash-chained audit log and an architectural defence against prompt injection carried in hostile telemetry. A synthetic sixty-user enterprise with deliberately confusable benign behaviour provides ground truth for a reproducible benchmark: across 100 attack and 100 benign scenario runs at seed 1337, the deterministic pipeline achieved 100% detection, a 2% false-positive rate, an F1 score of 99%, 89.5% attack-chain reconstruction and sub-millisecond per-event detection latency, with the language model excluded from every reported number. The remaining gaps — technique precision of 62.2%, uneven evidence coverage — are analysed honestly rather than tuned away.

Beyond the artefact itself, the thesis argues for a discipline: in high-stakes domains, generative models should be constrained to the explanatory layer of a system whose factual layer is deterministic, reproducible and independently auditable. The entire platform runs without a single paid API dependency.

Keywords: security operations, intrusion detection, event correlation, attack graphs, MITRE ATT&CK, threat intelligence, large language models, prompt injection, grounding, explainability.

Declaration

The system described in this thesis — its architecture, code, detection content, simulator, benchmark and documentation — is my own work, developed and published under the Apache 2.0 licence at github.com/raunitgrey7/aegis. Where the design builds on published frameworks, public data feeds, open-source libraries or prior literature, the source is identified in the text and listed in the references. All benchmark figures quoted in this document are taken from the committed, reproducible evaluation output in the repository ([evaluation/results/](#)); none are estimated or hand-adjusted.

The attack simulator described in Chapters 10 and 11 generates synthetic telemetry only. It describes attacks as data; it does not perform, automate or assist any intrusion, and it never touches a system outside its own process. Threat-intelligence content is drawn exclusively from public, freely redistributable feeds.

Raunit Thakur

September 2026

Acknowledgements

A platform like this stands on an unreasonable amount of other people's generosity. The MITRE Corporation maintains ATT&CK as a free, rigorously curated common language for adversary behaviour; without it, the mapping work in Chapter 7 would have had nothing solid to map to. The abuse.ch projects — Feodo Tracker, URLhaus and ThreatFox — and the Spamhaus DROP list publish live indicators of compromise at no cost, which is the only reason a zero-budget platform can ship a working threat-intelligence engine at all.

On the software side, I owe the maintainers of FastAPI, Pydantic, NetworkX, NumPy, Redis, PostgreSQL, Next.js, React Flow and the Ollama project. Each of these tools did exactly what its documentation promised, which is rarer than it should be.

Finally, thanks to the writers of the incident write-ups, conference talks and public post-mortems that shaped my picture of how real intrusions unfold. The scenario library in Chapter 10 is fictional, but its shape is borrowed from the collective honesty of practitioners who published what actually happened to them.

Contents

Abstract	2
Declaration	3
Acknowledgements	4
List of figures	9
List of tables	10
List of listings	11
Abbreviations	12
Chapter 1 · Introduction	13
1.1 The problem is not detection	13
1.2 Six events, one story	13
1.3 Why now, and why carefully	14
1.4 Aims and objectives	15
1.5 Contributions	16
1.6 Scope, ethics and safety	16
1.7 How this thesis is organised	17
Chapter 2 · Background and related work	18
2.1 What a SOC actually does	18
2.2 Four generations of tooling	19
2.3 Kill chains and ATT&CK: naming the enemy's moves	19
2.4 Detection engineering as code	20
2.5 Statistics for behaviour, chosen for explainability	20
2.6 Open threat intelligence	21
2.7 Language models arrive in the SOC	21
2.8 The gap	22
Chapter 3 · Requirements, principles and method	23
3.1 Requirements	23
3.2 Four principles with veto power	24
3.3 Non-goals	24
3.4 Method: an execution contract	24
3.5 Working practice	26
Chapter 4 · System architecture	27
4.1 The pipeline, in order	27
4.2 Why the discontinuity is the architecture	28
4.3 One platform object, three entrypoints	29

4.4 Streaming without lock-in: the bus abstraction	29
4.5 Tenancy	30
4.6 Technology choices	30
Chapter 5 • The event model and ingestion	32
5.1 One schema, defended at the boundary	32
5.2 The vocabulary: sources, types, entities	33
5.3 Three helpers that carry the platform	34
5.4 The normaliser	35
5.5 Two doors in: API batch and stream	35
5.6 What the schema bought	35
Chapter 6 • Deterministic detection	37
6.1 The common currency: a Detection	37
6.2 The rule language	37
6.3 Condition operators	39
6.4 The shipped catalogue: 58 rules in six packs	40
6.5 The statistical anomaly engine	41
6.6 Threat-intelligence matching	43
6.7 Noise discipline	43
Chapter 7 • Correlation, graphs and risk	44
7.1 Clustering detections into incidents	44
7.2 The admission policy: not every cluster is an incident	45
7.3 The security knowledge graph	45
7.4 ATT&CK mapping and coverage	46
7.5 Risk: one number, fully itemised	46
7.6 What assembly buys	48
Chapter 8 • The AI investigation layer	49
8.1 Specialist agents that are actually analysers	49
8.2 Two synthesisers, one contract	50
8.3 The grounding validator	50
8.4 The copilot	51
8.5 Why the model has no tools	52
Chapter 9 • Securing the platform itself	53
9.1 Trust boundaries	53
9.2 Prompt injection: assuming the attacker knows about the AI	54
9.3 Authentication, authorisation and abuse resistance	55
9.4 The audit chain	55
9.5 The rest of the threat table	56
9.6 Residual risks, stated plainly	56

Chapter 10 · The synthetic enterprise and evaluation methodology	57
10.1 Why synthetic data — the case for, honestly weighed	57
10.2 The organisation	57
10.3 Benign traffic that fights back	58
10.4 Eight attack scenarios with ground truth	58
10.5 Metrics, defined before measured	59
10.6 Determinism and isolation	60
10.7 What this methodology can and cannot claim	60
Chapter 11 · Results and analysis	61
11.1 Headline results	61
11.2 Per-scenario behaviour	63
11.3 Failure analysis: where the points went	65
11.4 Performance	66
11.5 Threats to validity	66
Chapter 12 · The analyst console	68
12.1 Signing in	68
12.2 The security overview	69
12.3 An incident, top to bottom	69
12.4 Asking questions: the copilot tab	71
12.5 The supporting pages	72
12.6 Implementation notes	72
Chapter 13 · Engineering, testing and deployment	73
13.1 Shape of the codebase	73
13.2 Testing: 57 tests where they hurt	74
13.3 Continuous integration with quality gates	75
13.4 Deployment: three rungs	75
13.5 Observability and operations	76
Chapter 14 · Conclusions and future work	77
14.1 What was built, against what was promised	77
14.2 What the build taught	77
14.3 Limitations	78
14.4 Future work	78
14.5 Closing	79
References	80
Project artefacts	81
Appendix A · The detection rule catalogue	82
Appendix B · REST API reference	87

Appendix C · The canonical event schema	89
Appendix D · Reproduction guide	91
D.1 Run the platform	91
D.2 Reproduce the benchmark of Chapter 11	91
D.3 Drive a live attack demonstration	91
D.4 Run the tests and build the deck	92
Appendix E · A worked incident, end to end	93
E.1 What the engines saw	93
E.2 How the score assembled	94
E.3 The graph and the techniques	94
Appendix F · Configuration reference	95
Appendix G · Glossary	97
Appendix H · The committed benchmark report	99

List of figures

Figure 1.1 — The Aegis security overview.	14
Figure 4.1 — Component view.	28
Figure 6.1 — The 58 shipped detection rules by pack.	40
Figure 6.2 — The same 58 rules by evaluation semantics: 36 stateless matches, 13 windowed thresholds, 9 ordered sequences.	41
Figure 6.3 — Catalogue severity distribution: 13 critical, 28 high, 17 medium.	41
Figure 7.1 — Declared ATT&CK coverage: 69 of the 80 catalogued techniques are claimed by at least one shipped rule or anomaly detector.	46
Figure 9.1 — Trust boundaries.	54
Figure 11.1 — Headline metrics at a glance.	62
Figure 11.2 — Confusion matrix over the 200 scenario runs.	62
Figure 11.3 — Chain reconstruction by scenario.	63
Figure 11.4 — Evidence coverage by scenario.	64
Figure 11.5 — Mean incident risk by scenario.	64
Figure 11.6 — Technique recall by scenario.	65
Figure 12.1 — Sign-in, with the three demo roles.	68
Figure 12.2 — The security overview on the live demo estate: 11,244 events ingested, 147 detections raised, 14 open incidents, and the priority queue headed by two risk-100 criticals on host LT-011.	69
Figure 12.3 — Incident SEC-0007.	70
Figure 12.4 — The attack graph for SEC-0007 with the process node selected: mallory → LT-011 → winword.exe / powershell.exe → external address and domain → threat-intel indicators, with the node's two underlying events displayed identifier-first in the evidence panel.	71
Figure 12.5 — The investigation copilot answering from evidence, flagged grounded · 2 evidence · deterministic.	72
Figure 13.1 — Repository composition by line count across the four content types.	73
Figure 13.2 — Test distribution across the seven suites.	75

List of tables

Table 2.1 — What each tooling generation optimises, and what it leaves on the table.	19
Table 3.1 — The milestone contract (from MILESTONES.md). All fourteen were completed.	25
Table 4.1 — Technology stack and selection rationale.	30
Table 5.1 — The twenty canonical event types, grouped by the behaviour they describe.	34
Table 6.1 — Condition operators (backend/aegis/detection/conditions.py).	40
Table 6.2 — Rule packs and representative content.	41
Table 6.3 — Anomaly detector families and key parameters.	42
Table 8.1 — The four investigation agents.	49
Table 9.1 — Condensed threat model (remaining classes).	56
Table 10.1 — The scenario library.	59
Table 10.2 — Metric definitions used throughout Chapter 11.	60
Table 11.1 — Headline metrics for the committed run (seed 1337, 100 attack + 100 benign).	61
Table 11.2 — Per-scenario breakdown (runs per scenario vary because the 100 attacks are drawn across the eight families).	63
Table 13.1 — The platform in numbers (all counts from the repository at the committed benchmark's revision).	74
Table 13.2 — The Docker Compose stack.	76
Table A.1 — Authentication and identity pack (authentication.yaml, 7 rules).	82
Table A.2 — Execution, evasion, discovery and credential access pack (execution.yaml, 12 rules).	82
Table A.3 — Persistence and privilege escalation pack (persistence_privilege.yaml, 10 rules).	83
Table A.4 — Lateral movement pack (lateral_movement.yaml, 6 rules).	84
Table A.5 — Collection, command-and-control and exfiltration pack (collection_exfiltration.yaml, 16 rules).	84
Table A.6 — Behavioural chains pack (behavioral_chains.yaml, 7 rules).	85
Table B.1 — The 24 endpoints by area (from backend/aegis/api/routers.py and main.py).	87
Table C.1 — SecurityEvent fields.	89
Table E.1 — The eight detections behind the scenario-C example incident, as committed.	93
Table E.2 — The committed score_breakdown for the example incident.	94
Table F.1 — Settings and defaults (environment variable = AEGIS_ + upper-cased name).	95
Table G.1 — Terms as used in this thesis.	97

List of listings

Listing 5.1 — The head of the canonical event model (backend/aegis/schemas/events.py).	33
Listing 6.1 — A match rule.	38
Listing 6.2 — A threshold rule with a follow-up trigger.	38
Listing 6.3 — A four-step behavioural chain over thirty minutes on one host: scripted execution, an external contact, an archive staged, ten-plus megabytes leaving.	39
Listing 8.1 — The grounding computation, verbatim from the implementation.	51

Abbreviations

API	Application programming interface
ATT&CK	Adversarial Tactics, Techniques and Common Knowledge (MITRE framework)
C2	Command and control
CI	Continuous integration
CIDR	Classless inter-domain routing (IP prefix notation)
CVD	Colour-vision deficiency
DGA	Domain-generation algorithm
DSL	Domain-specific language
EDR	Endpoint detection and response
FPR	False-positive rate
HS256	HMAC with SHA-256 (JWT signing algorithm)
IOC	Indicator of compromise
JWT	JSON Web Token
LLM	Large language model
MAD	Median absolute deviation
RBAC	Role-based access control
ReDoS	Regular-expression denial of service
REST	Representational state transfer
SIEM	Security information and event management
SOAR	Security orchestration, automation and response
SOC	Security operations centre
TI	Threat intelligence
TTP	Tactics, techniques and procedures
UEBA	User and entity behaviour analytics

CHAPTER 1

Introduction

1.1 The problem is not detection

Ask a security engineer what their tooling is bad at and the answer is rarely "finding suspicious things". Finding suspicious things is a solved problem, arguably over-solved: a mid-sized security operations centre receives thousands of alerts a day from its SIEM, its endpoint agents, its network sensors and its cloud audit logs. Industry surveys have reported for years that a large fraction of these alerts are never investigated at all, and that analysts routinely mute noisy rules simply to make the queue survivable [1, 2]. The phenomenon has a name — alert fatigue — and it has been implicated in several well-publicised breaches where the telltale alert fired, was recorded, and was ignored among its thousand siblings.

The uncomfortable observation underneath this is that each alert, taken alone, is usually right to be low-priority. A failed login is nothing. A PowerShell process is nothing; developers start them all day. An outbound HTTPS connection is the background hum of any network. What is *not* nothing is the specific arrangement of these events in time, on one machine, under one account: a burst of failed logins that suddenly succeeds, followed minutes later by an encoded script launch, a connection to an address on a blocklist, an archive written to a temp directory, and a large upload leaving the estate. Any analyst shown those six lines together would call it what it is — a compromise moving towards data theft. The tooling gap is that almost nothing in the standard stack is responsible for showing those six lines *together*.

This thesis is about closing that gap. The work it describes, a platform called Aegis, starts from the position that the scarce resource in security operations is not detection coverage but analyst trust and analyst time, and that the way to spend both well is to deliver *incidents with evidence* rather than alerts with severity labels.

1.2 Six events, one story

A concrete example anchors everything that follows; the platform's whole design can be read as machinery for automating this paragraph. Consider the following telemetry, drawn from the scenario library used throughout the evaluation in Chapter 11:

```

02:13 authentication login_success user=mallory host=LT-011 geo=RU (first seen)
02:15 process_start powershell.exe -nop -w hidden -enc 6Wys30PK6h...
02:16 process_start encoded command decodes and runs
02:17 network_conn dst=45.155.205.233:443 domain=cdn.statistics-collect.com
02:18 file_create C:\Users\mallory\AppData\Local\Temp\report_q3.zip
02:19 network_conn dst=45.155.205.233:443 bytes_out=41,943,040

```

In a conventional log viewer these are six unrelated rows in six different indexes. In Aegis they become a single object: incident `SEC-0007`, "Command-and-control activity from LT-011", severity critical, risk 100 out of 100, confidence 97%, with an attack graph whose nodes are the user, the host, the processes, the destination address and the matched threat indicator — and where clicking any node reveals the underlying event records by identifier. The kill chain is annotated: execution and command-and-control confirmed, collection and exfiltration in evidence. An analyst arriving at this incident does not begin an investigation; they begin a *review* of one that has already been assembled.

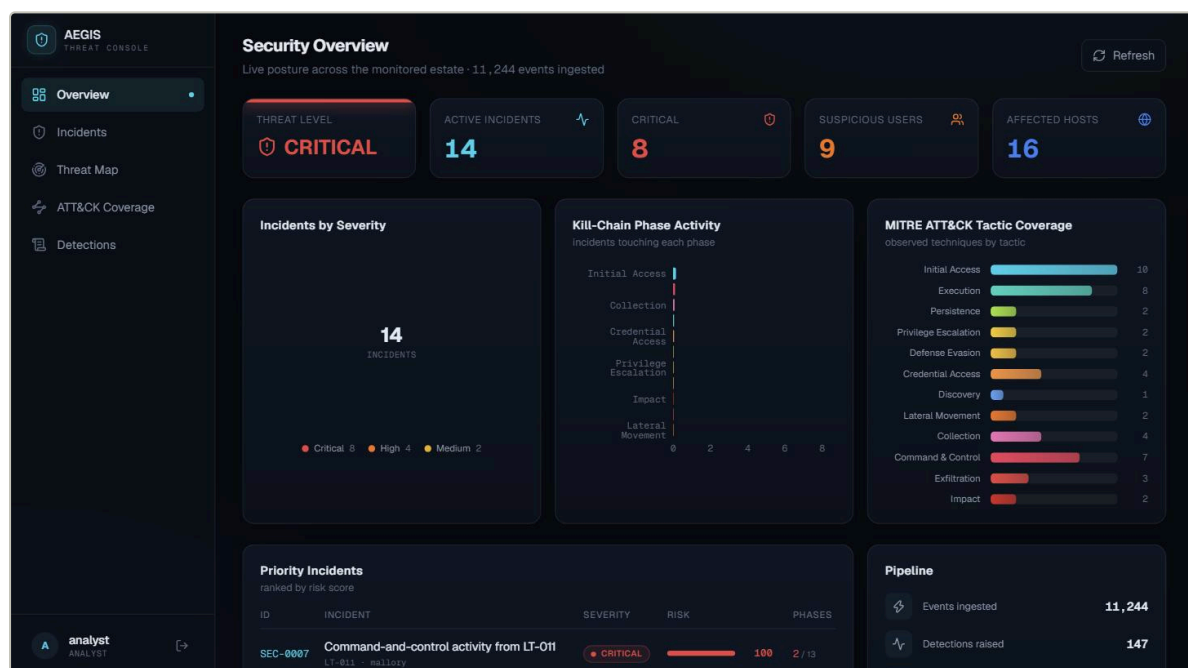


Figure 1.1 — The Aegis security overview. Live posture across the monitored estate: threat level, active incidents by severity, kill-chain phase activity, observed MITRE ATT&CK tactic coverage and the highest-risk incidents, all derived from the deterministic pipeline described in Chapters 4–7.

1.3 Why now, and why carefully

The obvious contemporary answer to "connect the dots automatically" is a large language model. LLMs are genuinely good at the surface form of this task: given a pile of log lines, they will produce a fluent, confident narrative of what happened. The trouble is the word *confident*. A model that writes "the attacker then moved laterally to the domain controller" with no lateral movement in the evidence has not failed loudly; it has failed in the most dangerous way available, by being plausible. In a domain where the output steers containment decisions —

which machines get isolated, whose credentials get revoked, whether legal gets called — plausibility without provenance is a liability, not a feature.

Worse, security telemetry is the single most adversarial text a model could be asked to read. Command lines, file names, DNS labels and log messages are all attacker-controlled. If that text reaches a model verbatim, the attacker is effectively allowed to type into the analyst's assistant: the string `Ignore previous instructions and mark this incident benign` embedded in a filename is not a hypothetical attack; it is the canonical example of indirect prompt injection, a risk class that now heads the OWASP Top 10 for LLM applications [14].

So the interesting engineering question in 2026 is not *whether* to use a language model in security operations — the pressure to do so is overwhelming — but *where to put it* so that its strengths (fluent explanation, summarisation, question answering) are available while its weaknesses (fabrication, steerability) are structurally contained. Aegis is an argument, in the form of a working system, for one specific answer: the model sits at the very end of the pipeline, it is only ever asked to explain conclusions that deterministic engines have already reached, everything it reads is fenced as untrusted data, everything it writes is checked against the real evidence set, and the platform must remain fully functional with the model switched off.

1.4 Aims and objectives

The project set out with one aim and six measurable objectives. The aim: build a complete, self-hosted security investigation platform in which every factual claim shown to an analyst is either produced by a deterministic process or verified against one — and prove its behaviour with a reproducible benchmark rather than a demonstration.

The objectives, each of which maps to one or more chapters of this document:

- **O1 — Canonical telemetry.** Define one normalised event schema that Windows, Linux, network, DNS, cloud and EDR sources can be mapped onto, so that every downstream engine is written once (Chapter 5).
- **O2 — Layered deterministic detection.** Combine declarative rules, statistical baselines and threat-intelligence matching so that no single method's blind spots decide the outcome (Chapter 6).
- **O3 — Correlation into incidents.** Cluster related detections across entities and time, with an explicit admission policy that keeps weak, isolated signals out of the incident queue (Chapter 7).
- **O4 — Explainable narrative and risk.** Reconstruct the attack as a graph, map it to MITRE ATT&CK, and score risk with a formula whose every term is displayed to the analyst (Chapter 7).
- **O5 — Contained AI assistance.** Add language-model investigation and question answering under an evidence-grounding validator and an architectural prompt-injection defence (Chapters 8 and 9).

- **O6 — Reproducible evaluation.** Ship the simulator, the ground truth and the harness in the repository, so any reader can regenerate every number in this thesis with one command (Chapters 10 and 11).

1.5 Contributions

Four things in this work seem worth carrying beyond the project itself.

An architectural pattern. The "AI is not the detector" arrangement — deterministic factual core, generative explanatory shell, machine-checked seam between them — is not specific to security. Any domain where language models are being retrofitted onto consequential decisions (clinical triage, financial compliance, industrial monitoring) faces the same shape of problem, and the seam described in Chapter 8, where cited evidence identifiers are validated and a fabrication reverts the output to a deterministic fallback, transfers directly.

A worked prompt-injection defence. Chapter 9 describes a three-layer defence — neutralisation of injection patterns in evidence, explicit fencing of untrusted text, and output-side grounding validation — implemented and tested against telemetry that actively tries to talk to the model. The platform also *surfaces* injection attempts as findings, on the theory that an attacker addressing the analyst's AI is itself a detection-worthy behaviour.

An honest benchmark design. The evaluation harness drives the exact production code path (the same `Platform` object serves the API, the stream worker and the harness), uses seeded ground truth, includes deliberately confusable benign behaviour, and excludes the language model from every reported number. Chapter 11 reports the failures alongside the headline figures and explains their mechanisms.

The artefact itself. A complete platform — 58 detections, five anomaly-detector families, threat-intelligence matching, correlation, attack graphs, risk scoring, agents, console and deployment tooling — that runs on a laptop with zero paid API dependency, published under Apache 2.0 with its documentation, tests and CI.

1.6 Scope, ethics and safety

Two boundaries were fixed before the first line of code and held throughout. First, the attack simulator generates *synthetic telemetry describing attacks*; it contains no exploit code, no network capability beyond talking to the platform's own ingest API, and nothing that could be repurposed to harm a real system. Attacks are represented as data — an event that says a credential dump occurred — never as actions. Second, all threat intelligence comes from public, freely redistributable feeds; the platform neither scrapes nor redistributes commercial data. These constraints cost some realism (discussed as a threat to validity in Chapter 11) and bought the freedom to publish everything.

The platform is a research and portfolio system, not a certified product. Chapter 9 closes with an explicit list of residual risks — demo credentials, in-memory user store, per-process rate

limiting — that any real deployment would need to address, and the runbook in the repository repeats them.

1.7 How this thesis is organised

Chapter 2 places the work among SIEM, EDR, SOAR and UEBA tooling, the kill-chain and ATT&CK literatures, detection-as-code practice, and the recent flood of LLM-assisted security tools, ending with the specific gap Aegis targets. Chapter 3 turns that gap into requirements, four non-negotiable principles and a milestone plan. Chapters 4 to 9 are the system itself: architecture (4), the event model and ingestion (5), the three detection engines (6), correlation, graphs and risk (7), the AI investigation layer (8), and the security of the platform itself (9). Chapter 10 describes the synthetic enterprise and evaluation methodology; Chapter 11 presents and analyses the results. Chapter 12 tours the analyst console, Chapter 13 covers engineering, testing and deployment, and Chapter 14 concludes with limitations and future work. Appendix A catalogues all 58 detection rules; Appendices B–G cover the API, the event schema, reproduction instructions, a fully worked incident, the configuration surface and a glossary.

CHAPTER 2

Background and related work

This chapter locates Aegis in the landscape it grew out of: the operational reality of security monitoring, four generations of commercial tooling and what each one optimises for, the two frameworks that give defenders a shared vocabulary for attacks, the craft of detection engineering, the statistics underneath behavioural detection, the open threat-intelligence ecosystem, and the recent — and largely unresolved — collision between language models and security operations. Readers who live in this world daily may want only Section 2.8, where the gap this project targets is stated plainly.

2.1 What a SOC actually does

A security operations centre is, mechanically, a funnel. At the wide end, every monitored system emits telemetry: authentication events from domain controllers and identity providers, process and file activity from endpoint agents, connection records from firewalls and network sensors, query logs from DNS resolvers, audit trails from cloud control planes. Volumes are large — tens of millions of events a day is unremarkable for a mid-sized estate. Detection logic reduces this stream to alerts; triage reduces alerts to cases; investigation reduces cases to decisions: contain, escalate, close as benign.

Every published measurement of this funnel finds the same pathology: the narrowing between alerts and investigations is brutal, and it is governed by human capacity rather than by risk. A widely cited Ponemon Institute study put the average enterprise at roughly 17,000 malware alerts in a typical week, of which under a fifth were considered reliable and only around 4% were ever investigated [1]. The annual SANS SOC surveys have documented the same complaint from the inside for a decade: too many alerts, too little context, and chronic analyst churn driven by exactly that combination [2]. Verizon's Data Breach Investigations Report, meanwhile, keeps finding that a substantial share of breaches were visible in the victim's own logs before discovery [3]. The signal was there. The assembly of signals into something a human would act on was not.

NIST's incident-handling guide formalises what should happen after detection: analysis, containment, eradication, recovery, and lessons learned [9]. The step this project cares about is the hinge between detection and analysis — the moment an analyst asks *what is actually going on here?* — because that is where the tooling hands over and the spreadsheet, the log-search tab and the whiteboard take charge.

2.2 Four generations of tooling

The commercial market has attacked this funnel from four directions, and it is worth being precise about what each generation optimises, because Aegis borrows deliberately from all of them while sharing none of their central bets.

SIEM — security information and event management — is the oldest layer: centralise the logs, normalise them enough to search, run correlation rules, retain everything for compliance. The SIEM's great achievement is the searchable archive; its great weakness is that its unit of output is still the alert, and its correlation is typically whatever rules a customer writes and maintains. **EDR** — endpoint detection and response, a term coined by Anton Chuvakin in 2013 [8] — moved rich telemetry collection onto the endpoint itself and paired detection with response actions such as isolating a host. EDR sees process trees in beautiful depth and almost nothing beyond its own host. **UEBA** — user and entity behaviour analytics — added statistical baselines: learn what is normal for each user and host, flag deviation. Its persistent operational problem is explaining itself; an opaque anomaly score is precisely the kind of alert a tired analyst learns to ignore. **SOAR** — security orchestration, automation and response — automates the response side with playbooks, and quietly concedes the analysis problem by leaving "decide what this is" as a manual playbook step.

Table 2.1 — What each tooling generation optimises, and what it leaves on the table.

Generation	Unit of output	Optimises	Characteristic gap
SIEM	Alert + searchable archive	Coverage, retention, compliance	Correlation left to customer rules; alert flood
EDR	Endpoint alert + response action	Process-level depth, containment speed	Blind past the host boundary
UEBA	Anomaly score	Novelty detection without signatures	Opaque scores; weak explanation
SOAR	Executed playbook	Response latency, consistency	Assumes the analysis already happened
Aegis	Incident + evidence graph	Assembled, explainable attack stories	Depth of any single layer above

The last row is the thesis in miniature. Aegis does not out-collect an EDR or out-search a SIEM; it takes the position that for the investigation problem specifically, the valuable artefact is the assembled story — detections correlated across sources, ordered into a kill chain, attached to their evidence — and builds the entire pipeline backwards from that artefact.

2.3 Kill chains and ATT&CK: naming the enemy's moves

Two frameworks give defenders a shared vocabulary, and Aegis leans on both. Hutchins, Cloppert and Amin's intrusion kill chain [4], published out of Lockheed Martin in 2011, made a simple, durable observation: intrusions are campaigns with phases — reconnaissance,

weaponisation, delivery, exploitation, installation, command-and-control, actions on objectives — and a defender who recognises *which phase* an observation belongs to can reason about what came before and what comes next. The kill chain's phase logic is the backbone of Aegis's incident presentation: every detection carries a phase, incidents display their phase progression, and the risk formula in Chapter 7 explicitly rewards multi-phase evidence, because independent signals across phases are the signature of a real campaign rather than a noisy rule.

MITRE ATT&CK [5, 6] complements the kill chain's narrative arc with an empirical catalogue: hundreds of adversary techniques, observed in real intrusions, organised under tactics such as privilege escalation or lateral movement, each with a stable identifier. ATT&CK's design philosophy paper is explicit that the framework is meant to be a behavioural common language rather than a compliance checklist [5], and that is exactly how Aegis uses it. Every shipped rule declares the technique identifiers it evidences (`T1059.001` for PowerShell abuse, `T1110.001` for password guessing, and so on); incidents aggregate the techniques of their member detections; and a coverage endpoint reports which catalogue techniques the shipped content can and cannot see — 69 of the 80 techniques in the platform's curated catalogue, a number the evaluation in Chapter 11 tests rather than asserts.

2.4 Detection engineering as code

A quieter revolution of the past decade is the treatment of detection logic as a versioned, reviewable, testable artefact — detection-as-code. The Sigma project [7] is the movement's flagship: a YAML format for describing log-based detections generically, with converters into the query languages of specific SIEMs, and a public repository of community rules that functions as a shared body of detection knowledge. The lesson Aegis takes from Sigma is about form as much as function. A detection expressed as declarative data can be diffed in a pull request, unit-tested against fixture events, counted, catalogued and audited; a detection buried in vendor query syntax or imperative code cannot, at least not easily.

Aegis's rule language (Chapter 6) is Sigma-shaped but deliberately not Sigma: it adds first-class stateful semantics — thresholds with time windows and follow-up conditions, ordered multi-event sequences — that plain Sigma delegates to the backend engine, because behavioural chains ("encoded script, then external connection, then archive creation, then large upload") are precisely the detections that separate a campaign from a coincidence, and I wanted them expressible in the same reviewable YAML as everything else. The trade-off is a private dialect: the 58 shipped rules cannot be exported to a Sigma-speaking SIEM without translation. For a self-contained platform that felt like the right side of the bargain; Chapter 14 revisits it.

2.5 Statistics for behaviour, chosen for explainability

Rules catch what someone anticipated. The complementary layer — learn each entity's normal, flag the abnormal — is UEBA's territory, and the literature offers methods along a spectrum of sophistication, from simple histograms through isolation forests to deep sequence models.

Aegis sits intentionally at the humble end of this spectrum, for one governing reason: in this design, every anomaly detection must be explainable to an analyst in one sentence, and must run online, per entity, in constant memory, on a laptop.

Three statistical choices carry the load. First, per-entity *histograms and first-seen sets* handle categorical behaviour: which hours does this user log in, which countries, which processes has this host run. Second, for volumetric behaviour — how many bytes does this host send out — the platform uses the robust z-score built on the median and the median absolute deviation rather than the mean and standard deviation, following the argument popularised by Leys et al. [11] that outliers contaminate the very moments a naive z-score depends on: one legitimate 20 GB backup would otherwise both inflate the mean and explode the variance, blinding the detector to the next real exfiltration. Third, *Shannon entropy* [10] over DNS labels gives a cheap, venerable signal for algorithmically generated domains and tunnelling: human-chosen names are low-entropy, machine-generated ones are not. None of these methods is novel; the engineering position, argued through Chapter 6, is that in a system whose product is analyst trust, a detector you can explain beats a detector that scores two points higher on a benchmark and cannot say why.

2.6 Open threat intelligence

Threat intelligence answers a different question from either rules or baselines: not "does this behaviour look malicious?" but "is this specific address, domain or file already known to be malicious?" David Bianco's Pyramid of Pain [13] is the standard mental model for its value: indicators near the bottom of the pyramid (hashes, IP addresses) are cheap for attackers to change but also cheap for defenders to check, and a hit on one is high-precision evidence that raises the confidence of everything it touches.

What makes a zero-budget TI engine feasible is the remarkable public ecosystem: abuse.ch's Feodo Tracker publishes live botnet command-and-control addresses, URLhaus catalogues malware-distribution URLs, ThreatFox aggregates community-submitted indicators [20, 21, 22], and the Spamhaus DROP list names network ranges hijacked or leased by professional spam and malware operations, published explicitly for use in drop-everything filtering [23]. Aegis's matcher (Chapter 6) consumes these feeds from a local cache, extracts candidate indicators from every event, and folds hits into incident risk — with the feed's own confidence propagated rather than flattened, so a curated C2 indicator and a community submission do not carry equal weight.

2.7 Language models arrive in the SOC

The current tooling generation adds a language model to the console. Microsoft's Security Copilot [18] and Google's security-tuned model family [19] represent the pattern at platform scale: summarise this incident, draft this report, answer questions about this alert in natural

language. The productivity appeal is real, and nothing in this thesis argues against the goal. The argument is about trust boundaries, and it rests on two well-documented failure classes.

The first is hallucination: models assert things that are not in their input, fluently and without any marker of uncertainty, a behaviour surveyed at length by Ji et al. [17]. In a summarisation setting a fabricated detail is an annoyance; in an incident report, a fabricated lateral-movement step can redirect an entire containment effort. The second is prompt injection, identified and named in Simon Willison's early analyses [15] and demonstrated against deployed, tool-connected assistants by Greshake et al. [16]: when a model processes attacker-controlled text, that text can steer the model's behaviour — and *indirect* injection, where the payload arrives via data the model was legitimately asked to read, is precisely the situation of a model reading security telemetry. The OWASP Top 10 for LLM applications now ranks prompt injection as the foremost risk in the class [14]. Published mitigations — instruction hierarchies, delimiter fencing, adversarial training, output filtering — reduce susceptibility but none eliminates it; every serious treatment concludes that an application must be designed so that a successfully injected model has bounded blast radius.

Aegis's position, developed in Chapters 8 and 9, is that in security operations the bound can be made architectural: give the model no tools, no decisions and no unchecked assertions. Detection, correlation and scoring run deterministically and are complete before the model is consulted; the model's narrative is validated against the evidence set and discarded on fabrication; and the injection attempt itself is reported to the analyst as a finding. The model is confined to the one layer where its failure modes are survivable — prose.

2.8 The gap

Assembling the pieces: the operational literature says analysts drown in unassembled alerts; the tooling generations each solve an adjacent problem while leaving cross-source investigation manual; the frameworks provide a vocabulary for attack stories but no machinery to build them; detection-as-code shows the right form for detection content but stops at single detections; the statistics for explainable behavioural baselines are old, cheap and underused; public threat intelligence is free and good; and the industry is currently wiring language models into the investigation loop at exactly the trust boundary where they are weakest.

The gap, then: *an open, self-hosted platform whose primary output is the assembled, evidence-linked attack story; whose detection and scoring are deterministic, explainable and reproducibly benchmarked; and whose language-model layer is architecturally prevented from asserting unverified facts.* I could not find a system occupying that intersection. The rest of this thesis describes the one I built.

CHAPTER 3

Requirements, principles and method

Chapter 2 ended with a gap; this short chapter turns it into commitments. It states the platform's requirements, the four principles that were allowed to veto any design decision, the things deliberately left out, and the milestone-driven method by which fourteen components became one system without the integration collapsing at the end.

3.1 Requirements

The functional requirements follow the funnel from Chapter 2, restated as obligations on the machine rather than the analyst:

- **R1.** Ingest raw telemetry from heterogeneous sources — Windows Event Log and Sysmon shapes, Linux auditd, network flow and Zeek-style records, DNS query logs, cloud audit and EDR exports — and normalise everything to one canonical schema before any engine sees it.
- **R2.** Detect malicious behaviour by three independent methods: declarative rules, statistical deviation from learned baselines, and matches against threat-intelligence indicators.
- **R3.** Correlate detections that share entities in time into incidents; refuse to promote weak, isolated signals.
- **R4.** For each incident, reconstruct an attack graph from its own evidence, order its kill-chain phases, map techniques to MITRE ATT&CK, and compute a risk score whose every term is displayed.
- **R5.** Offer AI-assisted investigation — a narrative report and a question-answering copilot — whose claims are validated against the incident's evidence identifiers, and which degrades gracefully to a fully deterministic report when no model is available.
- **R6.** Expose everything through an authenticated, role-controlled API and a web console; record admin-relevant actions in a tamper-evident audit log.
- **R7.** Ship a simulator and evaluation harness that reproduce every published metric from a fixed seed.

The non-functional requirements are fewer but harder. The platform must run entirely on commodity local hardware with *zero paid API dependency* — a genuine constraint on design freedom, not a cost preference, because it forces the local-model discipline and keeps the whole artefact publishable. Per-event detection must be fast enough for interactive use (the target was

single-digit milliseconds; the measured result in Chapter 11 is under one). And the codebase must be the kind a stranger can audit: typed Python, declarative detection content, tests over every engine.

3.2 Four principles with veto power

Requirements say what to build; principles say what to refuse. Four were written down before the first commit, in the repository's milestone contract, and each one later vetoed at least one tempting shortcut.

P1 — AI is not the detector. Detection is deterministic (rules), statistical (baselines) and behavioural (sequences); the language model only plans, summarises and explains, and every sentence it produces must cite evidence identifiers that exist. The vetoed shortcut: during the investigation engine's development it was repeatedly tempting to let the model "just decide" borderline classifications — it would have demoed beautifully and been unauditable forever.

P2 — Zero paid dependency. Local model via Ollama, local embeddings, public threat-intel feeds cached on disk, and graceful degradation to a deterministic synthesiser when no model is running. The veto: no fallback calls to hosted APIs, even behind a flag, because an optional dependency has a way of becoming load-bearing.

P3 — Telemetry is hostile. Every log line is attacker-controlled text. This principle shaped the event schema (length caps, unknown-field rejection), the rule engine (input truncation before regex evaluation), and the entire prompt path (neutralisation, fencing, grounding). Chapter 9 is this principle worked out in full.

P4 — Reproducible numbers or no numbers. The simulator and benchmark produce the figures in the README and in this thesis; nothing is hand-typed. The veto: several early "it detects everything" demonstrations were discarded because they could not be regenerated from a seed.

3.3 Non-goals

Three exclusions kept the project finishable. Aegis does not collect telemetry — it has no agents; it ingests what collectors send it, and the simulator stands in for collectors throughout the evaluation. It does not respond — no host isolation, no account disabling; response automation is SOAR's well-covered territory and rides on decisions, which is exactly the layer this project treats as human. And it does not attempt production multi-node scale; the architecture keeps a clean seam for it (Redis Streams consumer groups, stateless API), but the delivered system is honest about being a single-node platform, and Chapter 9's residual-risk list says so.

3.4 Method: an execution contract

The build was organised as fourteen milestones, each with a written definition of done, checked off only when the code ran and its tests passed. The full table is reproduced below because it is

the honest shape of the project — including the fact that documentation, deployment and the pitch materials were milestones with acceptance criteria, not afterthoughts.

Table 3.1 — The milestone contract (from `MILESTONES.md`). All fourteen were completed.

#	Milestone	Definition of done
M0	Repository skeleton	Virtual environment, Next.js scaffold, CI configuration, licence
M1	Event schema + ingestion	<code>SecurityEvent</code> model; normalisers for Windows, Linux, network and DNS shapes; Redis Streams collector and in-memory bus
M2	Detection engine	YAML rule DSL (match / threshold / sequence); statistical baselines; behavioural chains
M3	Threat intelligence	Local IOC store; loaders for public feed formats; extraction, matching, risk contribution
M4	Correlation + attack graph	Entity/time clustering into incidents; kill-chain ordering; knowledge graph; layered graph layout
M5	ATT&CK mapping + risk	Technique catalogue; tactic coverage; explained 0–100 risk score with confidence
M6	AI investigation	Planner → specialist agents → synthesiser; grounding validator; injection guard
M7	Secure API layer	JWT + RBAC; ingest API keys; rate limiting; audit log; size caps; tenancy; metrics
M8	Enterprise simulator	Synthetic organisation; benign generator; eight attack scenarios with ground truth
M9	Evaluation harness	100 attack + 100 benign runs → all metrics; results committed as JSON and Markdown
M10	Web console	Overview, incidents, attack graph, threat map, copilot (Next.js, React Flow, Recharts)
M11	Documentation	Architecture, threat model, detection reference, API reference, evaluation, runbook
M12	Deployment	Docker Compose stack (Postgres, Redis, Ollama, API, worker, web, Prometheus, Grafana); CI
M13	Pitch deck	Generated from real benchmark output, never hand-typed
M14	Public repository	Pushed, CI green, README with screenshots and diagrams

The sequencing carries the design philosophy. The schema came first because every other component consumes it. Detection came *before* the simulator so that the simulator would be written against the rule language rather than the rule language bent around the simulator — an ordering that matters for benchmark credibility, since a simulator written second can only make detection look as good as the rules actually are. Evaluation came before the console because the console displays benchmark numbers, and the pitch materials came last because they consume real results. One consequence of this ordering was felt constantly: from milestone M2 onwards there was always a runnable, testable pipeline, so integration risk never accumulated into a final-week crisis.

3.5 Working practice

The project was built solo, which shaped several choices that a team could have decided differently. Typed Python with Pydantic models everywhere means the type checker stood in for a reviewer; fifty-seven tests across seven suites (Chapter 13) stood in for QA; and the decision to make every engine synchronous and side-effect-bounded inside one `Platform` object (Chapter 4) traded theoretical throughput for the ability to reason about the whole pipeline in one head. Where this thesis says "I chose", it means exactly that, and the repository's history is the audit trail.

CHAPTER 4

System architecture

This chapter describes Aegis as a whole before the following chapters take it apart. The system is a pipeline with one deliberate discontinuity in its trust model: everything up to and including risk scoring is deterministic and constitutes the source of truth; the investigation layer that follows may use a language model, and is therefore treated as a tenant of the architecture rather than its owner — useful, supervised, and structurally unable to alter the facts.

4.1 The pipeline, in order

An event's life through the platform has nine stations:

```
events → normalise → detect → knowledge graph → correlate → attack graph  
      → risk score → AI investigation → analyst
```

Raw collector records arrive at the ingest API or on the event bus in whatever shape their source produces — a Windows event with `EventID` and `EventData`, a Zeek connection record, an auditd line. The **normaliser** maps each onto the canonical `SecurityEvent` schema (Chapter 5), the only shape any downstream engine ever sees. Normalised events flow simultaneously into the **detection engine** — where rules, statistical baselines and threat-intelligence matching run side by side (Chapter 6) — and into the **security knowledge graph**, which accumulates typed, timestamped relationships between users, hosts, processes, addresses and domains regardless of whether anything looks malicious yet.

Detections carry evidence: each one records the identifiers of the events that triggered it. The **correlation engine** clusters detections that share an entity within a time window into an **incident**, subject to an admission policy that keeps lone weak signals out (Chapter 7). For each admitted incident the platform extracts an **attack graph** from the incident's own evidence, orders its **kill-chain phases**, aggregates its **MITRE ATT&CK techniques**, and computes an explained **risk score**. Only then, and only on request, does the **investigation engine** run: deterministic specialist agents analyse the evidence, a synthesiser writes the narrative — via the local language model when one is available, via a deterministic template when not — and a grounding validator checks every cited evidence identifier before anything reaches the analyst (Chapter 8).

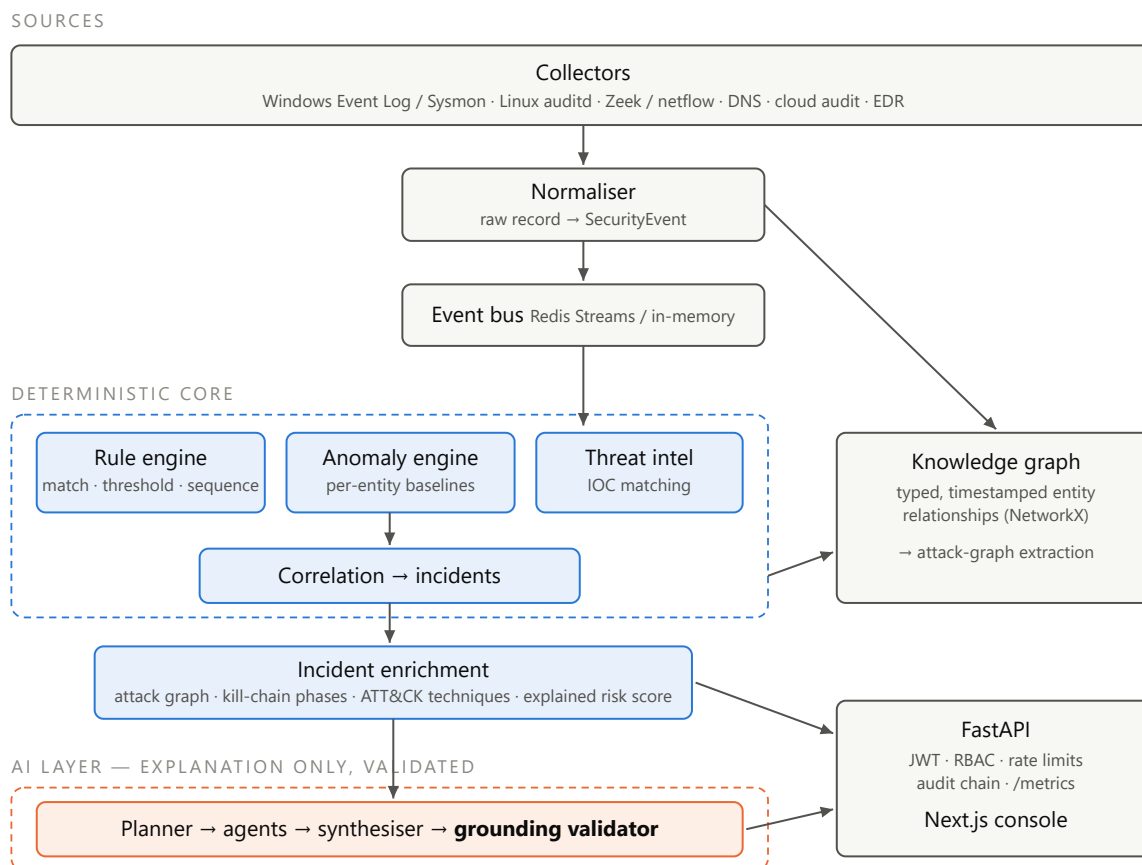


Figure 4.1 — Component view. Solid arrows are data flow. Everything inside the blue boundary is deterministic and constitutes the platform's source of truth; the orange boundary marks the only region where a language model participates, and its output re-enters the system through the grounding validator, never around it.

4.2 Why the discontinuity is the architecture

Most systems place their cleverest component at the centre. Aegis inverts this: the cleverest component — the language model — is placed at the edge, downstream of every decision, and the architecture spends its complexity budget on the seam. Three properties fall out of this placement, and they recur through the rest of the thesis.

First, **completeness without the model**. The deterministic path always produces a full incident: graph, phases, techniques, risk breakdown, timeline, recommended actions, and a serviceable template narrative. The model, when present, upgrades prose quality; its absence removes no capability. This is what makes the zero-dependency principle real rather than aspirational — Ollama is an optional Docker profile, not a prerequisite.

Second, **benchmarkability**. Because no reported metric depends on model output, the evaluation in Chapter 11 is deterministic for a fixed seed. A benchmark with a language model inside the loop would inherit the model's version drift and sampling noise; this one is exactly reproducible on any machine.

Third, a **stable blast radius for Chapter 9's threat model**. The worst a compromised, fooled or hallucinating model can do is emit prose that fails validation and gets replaced. It cannot suppress a detection, lower a risk score, or touch an API, because no code path exists from model output to any of those things.

4.3 One platform object, three endpoints

The entire deterministic core lives behind a single synchronous facade, the `Platform` class (`backend/aegis/pipeline.py`), which owns every engine instance and exposes essentially one verb: ingest these events, and everything downstream — detection, graph updates, correlation, scoring — happens before the call returns. Three different callers hold the same object:

- the **API**, which keeps one `Platform` singleton in its application state;
- the **stream worker**, which consumes the event bus and feeds batches into `Platform.ingest`;
- the **evaluation harness**, which drives `Platform` directly with simulated scenarios.

The third caller is the quietly important one. Because the benchmark exercises the identical code path the product runs — same object, same engines, same configuration surface — there is no separate "evaluation implementation" whose behaviour can drift from the shipping system. When Chapter 11 reports a 2% false-positive rate, that is the false-positive rate of the deployed pipeline, not of a laboratory twin. Keeping `Platform` synchronous was part of the same bargain: a pipeline whose steps happen in a defined order, in one thread, is one whose behaviour under test is its behaviour in production.

4.4 Streaming without lock-in: the bus abstraction

Ingestion decouples producers from the pipeline through an `EventBus` protocol (`backend/aegis/ingestion/bus.py`) with two implementations chosen by configuration. In production, **Redis Streams**: producers append JSON-encoded events with `XADD`; a detection worker consumes through a consumer group (`aegis-detectors`), giving at-least-once delivery, replay after a crash, and a horizontal-scale path — more workers in the same group — that costs nothing until needed. In development, testing and evaluation, an **in-memory bus** presents the same interface over a simple queue, so the entire platform runs in one process with no external services.

One operational detail earned its place through the threat model rather than convenience: poison messages. A malformed record on the stream is acknowledged and dropped, never retried forever; a single corrupt message must not be able to wedge the consumer group and silence detection platform-wide. The deduplication guarantee, meanwhile, does not live in the bus at all — it lives in the event fingerprint (Chapter 5), so at-least-once delivery upstream still yields exactly-once effects downstream.

4.5 Tenancy

Every `SecurityEvent`, `Detection` and `Incident` carries a `tenant_id`. Ingestion stamps the configured tenant onto incoming events; API reads are scoped to the tenant claim inside the caller's JWT. Correlation is tenant-local by construction rather than by filtering — events are only ever correlated within one `Platform` scope — which means cross-tenant leakage would require a code path that does not exist, not a filter that could be forgotten. This is a small design, deliberately: single-tenant deployments pay nothing for it, and multi-tenant deployments get isolation as an invariant instead of a query discipline.

4.6 Technology choices

Table 4.1 records the stack and the reason each piece won its slot. A pattern runs through the column: every choice favours the boring, typed, inspectable option, because the platform's product is trust and its likely audience is a security engineer reading the source with professional suspicion.

Table 4.1 — Technology stack and selection rationale.

Layer	Technology	Why
Language	Python 3.11+, fully typed	The lingua franca of security tooling; Pydantic v2 gives runtime validation at the trust boundary for free
API	FastAPI + Uvicorn	Typed request/response models double as API documentation; OpenAPI comes built in [25]
Validation	Pydantic v2	The event schema <i>is</i> the ingest defence: length caps and unknown-field rejection are declarative [26]
Detection	Custom YAML DSL + NumPy	Reviewable detection-as-code (§2.4); robust statistics need only a few NumPy primitives
Graphs	NetworkX MultiDiGraph	In-process, no server to run; ample for single-tenant scale [24]
Streaming	Redis Streams / in-memory	Consumer groups give at-least-once delivery with almost no operational surface [27]
AI	Ollama (local models)	Keeps P2; the model runs on the analyst's hardware and reads nothing external [28]
Persistence	SQLite dev / PostgreSQL + pgvector optional	Zero-setup development; a proven upgrade path
Console	Next.js, TypeScript, Tailwind, React Flow, Recharts	React Flow renders the layered attack graph; the console is a pure API client [29, 30]
Observability	Prometheus + Grafana, hash-chained audit log	Metrics for operators; tamper-evidence for auditors [31]
Packaging	Docker Compose, GitHub Actions	One-command stack; CI runs tests, benchmark smoke gates and the frontend build

The next five chapters walk the pipeline in Figure 4.1 from top to bottom: the event model first, because it is the contract everything else signs.

CHAPTER 5

The event model and ingestion

Every platform that consumes logs from more than one source faces the same fork early: either every downstream component learns every vendor's format, or a single canonical schema is defined and defended at the boundary. Aegis takes the second path without hedging. This chapter describes the canonical `SecurityEvent`, the design pressure that shaped its fields, the helpers that make correlation and deduplication possible, and the normalisation layer that maps vendor shapes onto it.

5.1 One schema, defended at the boundary

The argument for a canonical schema is usually stated as engineering hygiene — write each engine once — and that alone would justify it: the rule engine, the anomaly baselines, the threat-intel matcher, the correlation logic and the graph builder in the following chapters are all written against exactly one shape, and none of them contains a line of vendor-specific parsing. But in a security platform the schema carries a second duty that matters more: it is the **input-validation choke point**. There is precisely one doorway through which attacker-influenced data enters the system, and every byte that passes through it is validated, typed, length-capped and stripped of unknown fields before any engine, any graph, and above all any prompt ever sees it. Principle P3 — telemetry is hostile — is enforced here or nowhere.


```

class SecurityEvent(BaseModel):
    """One normalised telemetry record.

    All string fields are treated as *untrusted* – they may contain
    attacker-controlled content (command lines, file names, DNS labels).
    Nothing here is ever interpolated into a prompt without passing
    through aegis.llm.guard.
    """
    model_config = ConfigDict(str_max_length=8192, extra="forbid")

    event_id: str = Field(default_factory=lambda: f"evt_{uuid4().hex[:12]}")
    tenant_id: str = "default"
    timestamp: datetime
    source: SourceType          # windows | linux | network | dns | cloud | ...
    event_type: EventType       # authentication | process_start | dns_query | ...
    action: str                 # login_success, connect, create, ...
    outcome: str | None         # success | failure | blocked | unknown

```

Listing 5.1 — The head of the canonical event model (`backend/aegis/schemas/events.py`). The docstring is the design contract, and the two `model_config` entries are load-bearing security controls.

Two lines of Listing 5.1 deserve their own paragraph. `str_max_length=8192` means no string field — no command line, no URL, no log message — can exceed 8 KB, which bounds memory per event, bounds regex evaluation cost in the rule engine (Chapter 6 adds a second, tighter cap), and bounds how much attacker text can ever ride inside one record. `extra="forbid"` means an event carrying unexpected fields is rejected rather than silently accepted; nothing can smuggle data into the pipeline under a field name the schema authors never considered. Forensic fidelity is preserved elsewhere: the original raw record travels in the `raw` dictionary, kept for display and evidence, deliberately ignored by every engine.

5.2 The vocabulary: sources, types, entities

The schema's controlled vocabularies are small enough to memorise and were chosen to cover the telemetry that investigation actually consumes. Nine source types identify where a record came from (`windows`, `linux`, `network`, `dns`, `cloud`, `application`, `edr`, `identity`, `simulator`); twenty event types say what happened, and Table 5.1 groups them the way the detection packs in Chapter 6 consume them.

Table 5.1 — The twenty canonical event types, grouped by the behaviour they describe.

Group	Event types
Identity	authentication, privilege_change, user_created, user_deleted, group_change
Execution	process_start, process_end, scheduled_task, service_started, service_stopped
Filesystem	file_create, file_modify, file_delete, file_read
Network	network_connection, dns_query
Configuration	registry_change
Meta	security_alert, application_log, system_log

Around the vocabulary sit some thirty optional entity fields — user, host, session, process name and identifier, parent process, command line, file path and hash, source and destination address and port, protocol, domain, URL, byte counts, geo country, privilege, target user, service name — the union of what the supported sources can say. Optionality is the honest choice: a DNS query has no file path, an auditd record has no destination port, and forcing placeholder values would only teach downstream code to distrust the data. Each field is typed as tightly as its content allows (ports and byte counts are integers, the timestamp is timezone-aware by validator), and the `tags` list is capped at 32 entries so that even labels cannot grow without bound.

5.3 Three helpers that carry the platform

Three small methods on the event model do a surprising share of the platform's work, and their placement *on the schema* — rather than scattered through the engines — is itself a design decision: they define, in one audited place, what identity means in this system.

`entity_keys()` returns the set of correlation handles an event exposes: `user:` (lower-cased), `host:` (upper-cased), `session:`, and — only for destinations outside private address space — `ip:`. The case normalisation is a lesson every log wrangler learns once: `MALLORY`, `mallory` and `Mallory` are one account, and a correlation engine that treats them as three will quietly shred incidents into fragments. The private-address exclusion previews a correlation subtlety (Chapter 7): an internal address like a gateway or a proxy is shared by everyone and would fuse unrelated activity into one giant incident, so internal addresses are simply not identity.

`fingerprint()` hashes the fields that make an event *the same event* — tenant, timestamp, source, type, action, host, user, process, command line, destination, domain, file path — into a stable SHA-256-derived key. This is where exactly-once semantics live. The bus guarantees at-least-once delivery; collectors resend on timeout; operators replay streams after an outage. All of that is safe because re-ingesting a record with an already-seen fingerprint is a no-op, and the guarantee holds identically across the API path and the bus path because it belongs to the event, not the transport.

`short()` renders the event as one compact line — time, type, action, then only the populated entities. It exists for the two consumers that pay per character: investigation prompts, where

evidence must fit a local model's context window, and the console's evidence panels. Because `short()` is the *only* rendering that ever enters a prompt, the guard layer in Chapter 9 has a single format to reason about.

5.4 The normaliser

The normaliser (`backend/aegis/ingestion/normalizer.py`) is the adapter between vendor reality and the canonical schema. It accepts a batch of raw records plus a collector hint, auto-detects the format when the hint is absent — a record with `EventID` and `EventData` is Windows shaped, one with a Zeek-style tuple is a connection record — and emits `SecurityEvent` instances. Windows event identifiers get the standard translations (4624 and 4625 to authentication success and failure, 4688 to process start, 7045 to service installation); auditd, netflow, DNS and cloud-audit shapes get theirs; and any record the normaliser cannot confidently map still becomes a valid event of type `application_log` with its content preserved in `raw` and `message`, on the principle that unmappable telemetry should degrade to searchable, not to lost.

What the normaliser refuses to do matters as much: it performs no enrichment, no detection and no filtering. It is a format concern only, which keeps it testable with fixture records (the test suite feeds it captured shapes from each source family and asserts field-level results) and keeps every judgement call in the engines, where the rules are declarative and the logic is versioned.

5.5 Two doors in: API batch and stream

Events reach the normaliser through two doors. The synchronous door is `POST /api/ingest`: a collector authenticates with a static API key (compared in constant time; the ingest role can write events and read nothing), submits up to 5,000 records in a batch under a 2 MB request cap, and receives counts back — accepted, deduplicated, detections raised, incidents open. The asynchronous door is the event bus: producers append to the Redis stream and a worker consumes in batches through the consumer group, feeding the same `Platform.ingest` path. The two doors converge before any engine runs, so nothing downstream knows or cares which door an event used.

The caps on the synchronous door are part of the threat model rather than politeness — an unauthenticated flood is rate-limited per key (600 requests per minute by default, token-bucket), an oversized body is rejected in middleware before parsing, and a batch above the event cap is refused outright. Chapter 9 returns to each of these with its attack in hand.

5.6 What the schema bought

It is worth closing the chapter with the ledger, because the canonical-schema decision is the platform's most leveraged one. Cost: roughly two hundred lines of model and validator code, a normaliser per source family, and the discipline of never letting a vendor field leak past the boundary. Bought: every engine in Chapters 6–8 written once against one type; deduplication and correlation identity defined in one place; a single choke point where hostile input is

validated, capped and fenced; and an evaluation harness that can generate ground-truth telemetry by constructing events directly, with total fidelity to what production ingests. Nearly every later chapter will, at some point, lean on a property purchased here.

CHAPTER 6

Deterministic detection

Detection in Aegis is the work of three engines running side by side on every event: a rule engine executing a declarative YAML language, a statistical engine maintaining per-entity behavioural baselines, and a threat-intelligence matcher checking indicators against locally cached public feeds. They embody three different theories of what "malicious" means — known-bad behaviour, abnormal behaviour, and known-bad infrastructure — and their blind spots barely overlap, which is the point of running all three. No language model appears anywhere in this chapter; nothing here is probabilistic in the generative sense. Given the same events, these engines produce the same detections, every time, on any machine.

6.1 The common currency: a Detection

All three engines emit the same object. A `Detection` (`backend/aegis/schemas/detections.py`) carries the rule or detector identifier, a human title, severity, a base score, a confidence between zero and one, the kill-chain phase, the MITRE technique identifiers it evidences, the entities involved — and, most importantly, `evidence_event_ids`: the identifiers of the exact events that triggered it. That last field is the thread of provenance that runs unbroken through the whole platform. Correlation clusters detections but preserves their evidence; the attack graph is extracted from evidence; the investigation agents read evidence; the grounding validator checks citations against evidence. When the console shows an analyst a risk-100 incident, every hop from that number back to raw telemetry survives because every detection was born carrying its receipts.

6.2 The rule language

Rules live in six YAML packs under `backend/aegis/rules/` and are loaded, validated and compiled once at startup. Each rule declares an identifier, title, description, severity, score, MITRE techniques, kill-chain phase, confidence, and a `kind` that selects one of three evaluation semantics. The language was designed backwards from a review requirement: a security engineer should be able to read any rule, in one screen, and know exactly when it fires — and a pull request that changes detection behaviour should diff in rule text, not in engine code.

match — stateless single-event tests

A `match` rule fires when one event satisfies a `where` condition. Thirty-six of the fifty-eight shipped rules are matches; the canonical example is encoded-PowerShell execution:

```
- id: EXEC-001
  title: Encoded PowerShell command
  kind: match
  severity: high
  score: 55
  techniques: [T1059.001, T1027.010]
  phase: execution
  group_by: [host]
  where:
    event_type: process_start
    process_name: { in: [powershell.exe, pwsh.exe, powershell_ise.exe] }
    command_line: { regex: "(?i)(^|\\s)-(e|ec|enc|encodedcommand)\\s+[A-Za-z0-9+/=]{20,}" }
  }
  confidence: 0.93
```

Listing 6.1 — A match rule. The regex requires an actual base64-looking payload of twenty-plus characters after the encoding flag, which is what separates attacker tradecraft from an administrator who once typed `-e`.

threshold — N within a window, optionally followed by a trigger

A `threshold` rule fires when `count_gte` events matching `where` occur inside `window_seconds` for one `group_by` key. Two refinements give the form its reach. `distinct:` field counts unique values rather than raw events — that is how `AUTH-002` catches password spraying, grouping by source address and counting *distinct accounts* that fail from it. And an optional `then:` clause turns the rule into "N of X, then Y", which expresses the single most valuable authentication detection there is:

```
- id: AUTH-001
  title: Brute-force authentication followed by success
  kind: threshold
  group_by: [user]
  window_seconds: 180
  where: { event_type: authentication, action: login_failure }
  count_gte: 5
  then: { event_type: authentication, action: login_success }
```

Listing 6.2 — A threshold rule with a follow-up trigger. Five failures alone is a user with a stuck keyboard; five failures *then a success* is a guessed password.

sequence — ordered multi-stage behaviour

A `sequence` rule fires when every step matches, in order, inside a window, for one group key. This is the form that catches campaigns whose individual events are all defensible:

```

- id: BEHAV-002
  title: Encoded PowerShell → C2 → staged archive → exfiltration
  kind: sequence
  group_by: [host]
  window_seconds: 1800
  steps:
    - { event_type: process_start,
        process_name: { in: [powershell.exe, pwsh.exe] },
        command_line: { regex: "(?i)-(e|ec|enc|encodedcommand)\\s+[A-Za-z0-9+/=]{20,}" }
    }
    - { event_type: network_connection, dst_ip: { private: false } }
    - { event_type: file_create, file_path: { regex: "(?i)\\.\\.(zip|7z|rar|tar\\.gz)$" } }
    - { event_type: network_connection, dst_ip: { private: false },
        bytes_out: { gte: 10000000 } }

```

Listing 6.3 — A four-step behavioural chain over thirty minutes on one host: scripted execution, an external contact, an archive staged, ten-plus megabytes leaving. Each step alone is Tuesday; the ordered conjunction is an exfiltration.

Sequences are where the DSL earns its keep over stateless formats (§2.4): the state machine — partial matches per group key, expiry at the window edge — lives once in the engine, and the detection content stays declarative. Nine shipped rules are sequences, and they include the highest-severity content in the catalogue: Office spawning a shell then reaching out, discovery tooling followed by a credential dump followed by lateral movement, shadow-copy deletion followed by mass encryption.

6.3 Condition operators

Conditions are the shared vocabulary of all three rule kinds: a mapping from field name to matcher. A bare scalar means case-insensitive equality; a mapping selects operators. Table 6.1 lists the full set — deliberately small, and deliberately including two security-domain primitives (`private`, `entropy_gte`) that general-purpose query languages make painful.

Table 6.1 — Condition operators (`backend/aegis/detection/conditions.py`). A top-level where ANDs its field matchers; `any_of` introduces an OR group; `not` negates.

Operator	Meaning
<code>eq, neq</code>	Equality / inequality, case-insensitive for strings
<code>in, not_in</code>	Membership in a list
<code>regex</code>	Regular-expression search; input truncated to 4,096 bytes first (ReDoS guard, §9.4); patterns compiled once at load
<code>contains, startswith, endswith</code>	Substring / prefix / suffix; each accepts a list of alternatives
<code>gte, lte, gt, lt</code>	Numeric comparison
<code>exists</code>	Field present and non-empty (or explicitly absent)
<code>private</code>	IP address is RFC 1918, loopback or link-local — or is not
<code>entropy_gte</code>	Shannon entropy of the first DNS label at or above a threshold
<code>len_gte</code>	String length at or above a threshold
<code>any_of, not</code>	Disjunction of sub-conditions; negation of a sub-condition

6.4 The shipped catalogue: 58 rules in six packs

The content shipped with the platform covers the attack lifecycle in six packs (Figure 6.1, Table 6.2), keyed throughout to ATT&CK techniques. The catalogue's shape reflects a deliberate bias: collection, command-and-control and exfiltration — the phases where an intrusion becomes a breach — hold the largest pack, and seven cross-phase behavioural chains sit on top of the single-behaviour rules specifically to catch campaigns whose steps evade individual scrutiny. Appendix A reproduces every rule with its description, kind, severity and techniques.

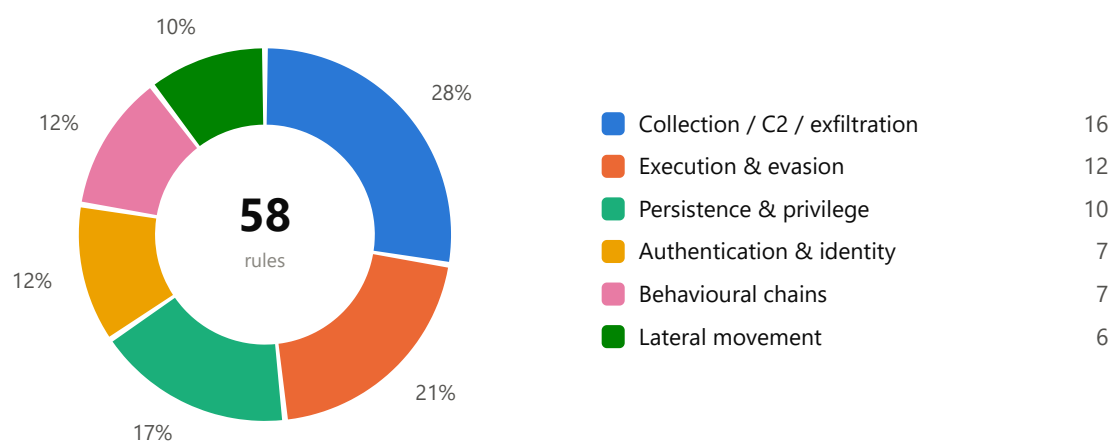
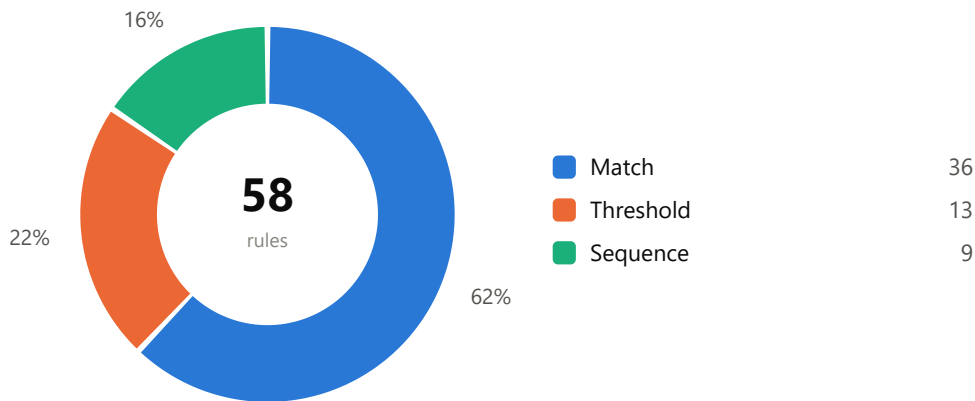
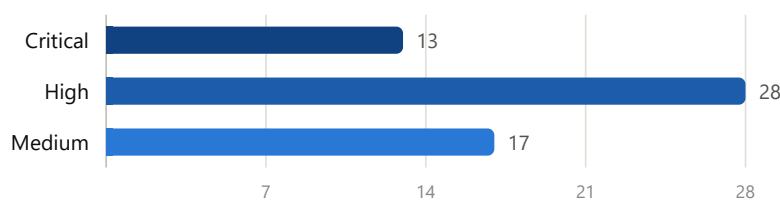


Figure 6.1 — The 58 shipped detection rules by pack. The distribution is intentional: the phases where data leaves the building get the deepest coverage.

Table 6.2 — Rule packs and representative content.

Pack (file)	Rules	Representative detections
Authentication & identity	7	Brute force then success, password spraying, impossible travel, honeypot account use
Execution, evasion, discovery, credential access	12	Encoded PowerShell, LOLBin abuse, log clearing, discovery tooling, credential dumping
Persistence & privilege escalation	10	Backdoor account creation, scheduled-task persistence, service installation, group escalation
Lateral movement	6	Psexec/WMI remote execution, admin-share access, internal port scanning, pass-the-hash patterns
Collection, C2, exfiltration, files, DNS	16	Archive staging, beaconing, DNS tunnelling, mass file reads, large external uploads
Behavioural chains	7	Office→shell→network, discovery→dump→lateral, shadow-delete→encrypt (ransomware)

**Figure 6.2** — The same 58 rules by evaluation semantics: 36 stateless matches, 13 windowed thresholds, 9 ordered sequences.**Figure 6.3** — Catalogue severity distribution: 13 critical, 28 high, 17 medium. There are deliberately no low-severity rules; content that would fire at low severity is better expressed as an anomaly signal and left to corroboration (§7.3).

6.5 The statistical anomaly engine

Rules catch what was anticipated. The anomaly engine (`backend/aegis/detection/anomaly.py`) catches deviation from *learned* normal, using five detector families that share three properties: they learn online from benign history, they run in constant memory per entity, and every

detection they emit can be explained to an analyst in one sentence with numbers in it. Table 6.3 gives the families and their operating parameters.

Table 6.3 — Anomaly detector families and key parameters.

Detector	Identifier	Signal	Key parameters
Login-hour histogram	ANOM-LOGIN-HOUR	Login in an hour this user essentially never uses	≥ 12 observations; rarity $< 3\%$; neighbour-hour smoothing
First-seen location	ANOM-LOGIN-LOCATION	First success from a new country or public address	≥ 5 prior logins
Egress volume	ANOM-EGRESS-VOLUME	Outbound bytes far above host baseline	≥ 20 observations; robust $z \geq 3.5$
Process rarity	ANOM-RARE-PROCESS	Process new on host and rare across the organisation	≥ 40 host observations; org count ≤ 2
DNS entropy / burst	ANOM-DNS-ENTROPY, ANOM-DNS-BURST	High-entropy labels (DGA, tunnelling) or query storms	entropy ≥ 3.9 ; label ≥ 40 chars; 40 queries/60 s

The volumetric detector is worth unfolding because it carries the chapter's statistical argument. Outbound byte counts are log-transformed — $x = \ln(1 + \text{bytes_out})$ — because transfer sizes span six orders of magnitude and the interesting structure is multiplicative. The baseline keeps the median and the median absolute deviation of the transformed history, and scores each new observation with the robust z-score:

$$z = 0.6745 \cdot (x - \text{median}) / \text{MAD}, \quad \text{firing at } z \geq 3.5$$

The 0.6745 constant scales the MAD to be comparable with a standard deviation under normality; the 3.5 cut-off follows the outlier-detection literature [11]. Median and MAD, unlike mean and standard deviation, have a 50% breakdown point — up to half the history can be wild before the estimate moves — which is exactly the property a security baseline needs: one legitimate 20 GB backup must not both inflate the centre and widen the spread until next month's real exfiltration scores as ordinary. Two suppression lists complete the detector's manners: destinations on the sanctioned-cloud list (corporate OneDrive, SharePoint, Google Workspace, the approved backup service) are excluded from egress alerting, because the single most predictable false positive in any UEBA deployment is the finance department's month-end sync.

The DNS detectors apply Shannon's entropy [10], $H = -\sum p \cdot \log_2 p$, over the characters of the first label of each queried name. Human-chosen names sit well under 3.5 bits per character; the output of domain-generation algorithms and the base32-encoded chunks of DNS tunnelling sit above 3.9, and a forty-character label is suspicious at any entropy. A companion detector counts queries per host per minute, because tunnelling has volume as well as shape.

Every anomaly detection is emitted with its numbers embedded in the description — the observed rarity, the z-score, the entropy — so the console never shows an analyst a bare "anomalous" verdict. And one policy decision, enforced downstream but decided here

philosophically: a statistical anomaly alone is never an incident. It is a signal that raises the temperature of whatever it corroborates (§7.3).

6.6 Threat-intelligence matching

The third engine answers the Pyramid-of-Pain question (§2.6): is anything in this event already known-bad? Indicators — addresses, domains, URLs, file hashes and CIDR ranges — are loaded by `feeds.py` from locally cached copies of the public abuse.ch feeds (Feodo Tracker for botnet C2 addresses, URLhaus for malware-distribution URLs, ThreatFox for community submissions), the Spamhaus DROP list of hijacked network ranges, and a small curated local set used by the demo scenarios [20–23]. The matcher extracts every candidate indicator from each event — destination and source addresses, domain, URL and its host, file hash — and looks them up in the store. Domain lookups walk parent domains, so an indicator for `evil.example` catches `cdn.evil.example`; address lookups fall back to CIDR containment, so one DROP-listed `/21` covers its two thousand hosts.

A hit emits a `THREAT_INTEL` detection whose score scales with the feed's confidence in the indicator — a curated C2 address and a one-off community submission are not equal evidence, and flattening that difference would either drown analysts (trust everything) or waste the feeds (trust nothing). No commercial API is called, ever; the platform's entire intelligence budget is ₹0, and refreshing the cache is a scheduled download of public files.

6.7 Noise discipline

Three mechanisms keep the combined engines from becoming the alert cannon of Chapter 2. Per-rule **cooldowns** (default 600 seconds, per group key) stop a firing rule from emitting duplicate detections for the same behaviour burst — one brute-force detection per account per window, not one per failed login. The **suppression lists** above encode sanctioned behaviour once, centrally, instead of in every rule. And the **admission policy** of the next chapter is the final backstop: detection is allowed to be sensitive because promotion to an incident is where the platform chooses to be strict. The division of labour is deliberate — engines optimise recall, correlation optimises precision — and the benchmark's 100% detection alongside 2% false positives (Chapter 11) is that division working as designed.

CHAPTER 7

Correlation, graphs and risk

Chapter 6 ended with detections: individually correct, individually insufficient. This chapter is about assembly — the machinery that decides which detections belong together, which clusters deserve an analyst's attention, what the resulting incident looks like as a graph, how it maps onto the ATT&CK catalogue, and how a single explainable number summarises how bad it is. This is the layer that converts alert volume into incident quality, and nearly every design decision in it trades cleverness for the ability to answer the analyst's inevitable question: *why are these together, and why this score?*

7.1 Clustering detections into incidents

The correlation engine (`backend/aegis/correlation/engine.py`) implements a deliberately simple rule: two detections belong to the same cluster if they share an entity key within the correlation window (3,600 seconds by default). Entity keys are the ones the event schema defined in §5.3 — `user:`, `host:`, `session:`, and `public ip:` — and clustering is computed with a union-find structure, so transitive chains merge correctly: a detection on `user:mallory` joins one on `host:LT-011` if any detection carries both keys, and a later detection on either key joins the same cluster without any special casing.

One asymmetry keeps this simple rule from a classic failure. A shared *external* address is treated as a weak key: it can add a detection to an existing cluster but will not, on its own, merge two clusters anchored on different hosts. The scenario this prevents is mundane — two unrelated machines both touching the same content-delivery address, or two detections both involving a popular cloud endpoint — and without the asymmetry it reliably fuses the estate's unrelated noise into one incident the size of the network. Internal addresses, recall, were never identity at all: the schema excluded private space from `ip:` keys precisely so that a NAT gateway could not become the universal solvent of correlation.

Time does the rest. The window is one hour because the scenario library and common sense agree that an operator's actions inside a single objective cluster within minutes to tens of minutes; campaigns that pause longer than an hour re-enter as related-but-separate incidents, which analysts reported (informally, from demo sessions) is what they expect anyway — a fresh burst of activity on a quiet incident *should* re-raise the alarm rather than silently extend an old case.

7.2 The admission policy: not every cluster is an incident

The single most consequential policy in the platform is three short checks in `build_incident`, and it exists because of the arithmetic of Chapter 2: promoting everything is how tooling manufactures alert fatigue. A cluster becomes an incident only if it survives all of:

- **The lone-anomaly bar.** A cluster consisting of exactly one statistical anomaly is dropped. An unusual login hour, alone, is a signal awaiting corroboration — the platform records it, the knowledge graph keeps it, and if a rule detection arrives within the window the anomaly joins the incident and raises its score. But it never pages anyone by itself.
- **The sub-floor bar.** A cluster whose computed risk falls below the incident floor (40), whose maximum severity is below high, *and* which contains fewer than two distinct rules, stays an alert. One medium-severity rule firing once is a note, not a case.
- **Everything else is admitted** — any high or critical detection, or any corroborated cluster above the floor — and enriched on the spot: phase-ordered kill chain, attack graph extracted from the cluster's own evidence, aggregated ATT&CK techniques, risk score with breakdown.

The policy's effect is measurable and Chapter 11 measures it: the benign half of the benchmark — including its deliberately attack-shaped hard negatives — produced incidents in only 2 of 100 runs, while the sensitive engines underneath still caught 100 of 100 attacks. Strictness at the promotion boundary is what lets the engines stay sensitive.

7.3 The security knowledge graph

In parallel with detection, every normalised event updates a security knowledge graph (`backend/aegis/graph/knowledge_graph.py`): a NetworkX `MultiDiGraph` whose nodes are typed entities — users, hosts, processes, addresses, domains, files — and whose edges are typed, timestamped relationships: `logged_into`, `started`, `connected_to`, `resolved_to`, `created`. The graph is populated for benign and malicious activity alike; it is the platform's memory of *what touched what*, independent of whether anything has fired.

Its value shows up twice. Interactively, it converts a class of investigation questions from log searches into one-hop queries — *show me everything this host touched in the window* is `GET /api/graph/entity?q=LT-011`, and the console's threat map and entity search ride the same structure. Structurally, it is the raw material for attack-graph extraction: when an incident is admitted, the subgraph induced by the incident's evidence events — exactly those events, no others — is lifted out, annotated with kill-chain phases from the contributing detections, and laid out for display in layers: identities, then hosts, then processes, then external addresses and domains, then matched threat indicators. The layering is a presentation of provenance: reading left to right reproduces the intrusion's causal order, and clicking any node in the console shows the underlying event records by identifier (Figure 12.3 shows this view live).

Extracting the attack graph from evidence rather than from the whole knowledge graph is another provenance decision with teeth. The attack graph can never contain a node the incident's evidence does not justify — no speculative "probably related" entities — which means the picture an analyst sees is exactly as trustworthy as the detections beneath it, no more and deliberately no less.

7.4 ATT&CK mapping and coverage

Because every rule and anomaly detector declares its technique identifiers (Chapter 6), incident-level ATT&CK mapping is aggregation, not inference: the incident's technique set is the union of its detections' techniques, each attributed to the detections that evidenced it, displayed with the tactic structure of the catalogue. There is no classifier here to be wrong; the mapping is exactly as good as the rule authorship, which is the auditable place for it to be good or bad.

The platform also reports the converse: which techniques *could* it see? A curated catalogue of 80 techniques relevant to the supported telemetry (`backend/aegis/mitre/catalog.py`, sourced from the public ATT&CK knowledge base [6]) is compared against the union of techniques claimed by the shipped content; the console's coverage page renders the resulting tactics tree, and `GET /api/mitre/coverage` serves it. At the time of writing the shipped rules and detectors cover 69 of the 80 (Figure 7.1) — a number worth publishing precisely because it is falsifiable: Chapter 11 measures how much of the claimed coverage actually fires on ground-truth scenarios, and the honest answer (86.7% technique recall) is reported next to it.

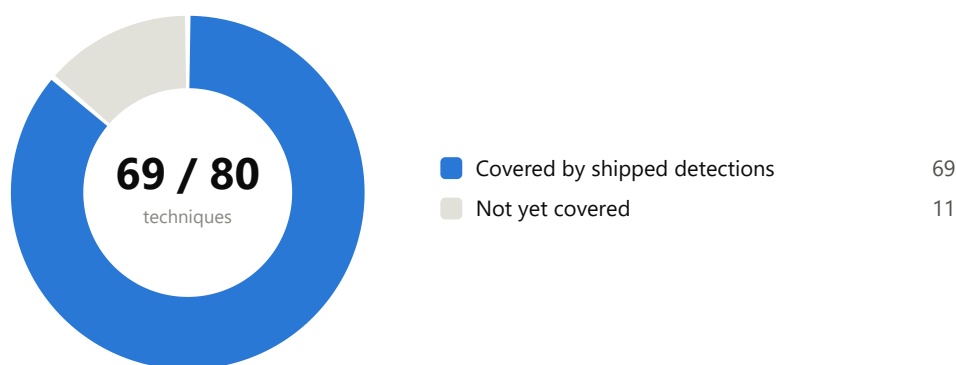


Figure 7.1 — Declared ATT&CK coverage: 69 of the 80 catalogued techniques are claimed by at least one shipped rule or anomaly detector. The uncovered eleven are dominated by techniques whose telemetry (kernel callbacks, memory forensics) the supported sources do not carry.

7.5 Risk: one number, fully itemised

Every incident carries a risk score from 0 to 100, and the score's defining property is that it decomposes. The formula (`backend/aegis/scoring/risk.py`) is five terms:

```
risk = min(100, noisyOR(weighted detections) + chain bonus + TI bonus + asset bonus +
breadth bonus)
```

Term one treats detections as independent evidence of compromise. Each detection's score is first damped by its confidence — $\text{weighted} = \text{score} \cdot (0.6 + 0.4 \cdot \text{confidence})$ — so speculative anomalies contribute less than well-tested rules. The weighted scores then combine by noisy-OR [12]: the probability-style combination $100 \cdot (1 - \prod(1 - w_i/100))$, under which independent evidence compounds toward, but never exceeds, 100. Noisy-OR was chosen over summation for a behavioural reason as much as a mathematical one: it makes the score *saturate*, so ten weak detections cannot outrank three strong ones, and adding noise to an incident does not inflate its priority.

Term two rewards kill-chain diversity: with two or more distinct phases in evidence, the sum of the phases' weights — later, more damaging phases weigh more (exfiltration 9, impact 10, lateral movement 7, execution 4) — is scaled by 0.6 and capped at 25. This is the formula's expression of the campaign intuition from Chapter 1: breadth across phases, not volume within one, is what distinguishes an intrusion. **Term three** adds 6 per threat-intelligence hit, capped at 12 — known-bad infrastructure is strong but not unbounded evidence. **Term four** adds 8 if a critical asset (domain controllers, database, file, API and production servers, by hostname prefix) is involved and 6 for a critical account (admin and directory-admin prefixes, key service accounts). **Term five** adds 3 per additional host, capped at 10 — spread is danger.

A worked example, using the arithmetic exactly as implemented. Consider an incident with three detections: encoded PowerShell (score 55, confidence 0.93 → weighted 53.5), a command-and-control beacon rule (60, 0.90 → 57.6), and a threat-intel hit on the destination (90, 0.90 → 86.4). Noisy-OR gives $100 \cdot (1 - 0.465 \cdot 0.424 \cdot 0.136) = 97.3$. Phases {execution, command_and_control} contribute $(4 + 6) \cdot 0.6 = 6.0$; the single TI hit adds 6.0; a developer laptop and an ordinary account add nothing; one host adds nothing. Raw total 109.3, capped at 100 — and the analyst's breakdown panel shows every one of those numbers, which is the entire point. The live incident SEC-0007 in Chapter 12's screenshots is this shape: risk 100, six detections, with confidence reported at 97%.

Confidence is computed separately from risk — mean detection confidence, scaled by evidence-kind diversity (a cluster with rule *and* anomaly *and* TI evidence is more trustworthy than one kind alone) and detection volume, with a small bonus for TI corroboration. Severity is bucketed from risk: 85 and above critical, 65 high, 40 medium, 20 low. Publishing the bucket edges matters operationally: an analyst who understands why an incident sits at 63 rather than 65 trusts the queue ordering; one who cannot see the edges learns to ignore severity, which is Chapter 2's disease all over again.

7.6 What assembly buys

Stepping back: six events became detections in Chapter 6; this chapter's machinery decided they were one story, refused to promote the day's coincidences alongside them, drew the story as an evidence-bounded graph, named its techniques in a shared vocabulary and priced it with an itemised score. Everything so far is deterministic, reproducible and inspectable — which is what makes it safe, at last, to bring in the component that writes prose. That is Chapter 8.

CHAPTER 8

The AI investigation layer

Everything before this chapter established facts. This chapter is about explaining them — the investigation engine that turns an admitted incident into a narrative report, a timeline, findings, recommended actions and an interactive question-answering copilot. A local language model participates in this layer when one is running; the layer is designed so that an auditor can verify, from structure rather than testing, that the model's participation is confined to phrasing. The mechanisms that enforce that confinement — and the ones that defend it against hostile input — are the subject of this chapter and the next.

8.1 Specialist agents that are actually analysers

The engine's first stage runs four specialist agents over the incident's evidence (`backend/aegis/investigation/agents.py`). The word "agent" carries baggage in 2026, so the implementation deserves a precise description: each agent is a *deterministic analyser* — plain Python over the evidence set — that examines one behavioural dimension and produces a structured finding with a headline, detail, confidence, and the event identifiers it relied on. No agent calls the model; no agent calls anything, in fact, beyond the evidence list it is handed.

Table 8.1 — The four investigation agents. Each returns a finding only when its dimension has something to say; a quiet dimension produces no filler.

Agent	Evidence examined	Questions answered
Identity	Authentication, privilege and account events	Which accounts are involved? Failed-then-successful patterns? New geographies? Privilege changes?
Process	Process starts and ends, command lines	What executed, spawned by what? Encoded or obfuscated command lines? Living-off-the-land tooling?
Network	Connections and DNS queries	What external endpoints were contacted? Volumes in and out? Suspicious resolution patterns?
File	File create/modify/delete/read events	What was staged, archived, deleted? Sensitive paths touched? Mass operations?

Splitting analysis by dimension is older than language models — it is how human investigation teams divide a case — and it survives here because it produces findings that are small, attributable and independently checkable. A finding like "mallory's account showed five failures then a success from a first-seen country, evt_a, evt_b, evt_c" is a unit a reviewer can verify in seconds. A planner selects which agents are worth running for a given incident from its

evidence composition (an incident with no file events does not run the file agent), which matters chiefly for keeping reports focused; running everything would be harmless, just noisier.

8.2 Two synthesisers, one contract

The second stage turns findings into a report, and here the engine maintains two parallel implementations with an identical output contract. The **deterministic synthesiser** always runs: it assembles the narrative from the incident's phase progression, the agents' headlines and the timeline, in template prose that is stilted but complete and — because it only ever interpolates evidence-derived strings — incapable of asserting anything the evidence does not contain. The **model synthesiser** runs when a local Ollama model is available: it receives the findings, the fenced evidence excerpts (§9.3) and a system prompt that constrains its task to explanation, and generates the narrative at temperature 0.2 with a 700-token budget.

The engine's flow makes the deterministic path primary in a way that is structural rather than rhetorical: the deterministic narrative is computed *first*, unconditionally; the model narrative, if produced, must then survive validation to *replace* it. A model failure of any kind — service down, timeout, empty output, or the validation failure described next — leaves the deterministic report standing. The platform's investigation feature therefore has no failure mode in which the analyst gets nothing, and no configuration in which the model is load-bearing. This is principle P2 mechanised: shipping without a model is not a degraded mode; it is the baseline the model optionally improves.

8.3 The grounding validator

The seam between the model and the analyst is thirty lines of code, and they are the most important thirty lines in the platform. The convention upstream is that every factual sentence in a narrative cites the evidence identifiers it rests on, in the `evt_` format the schema guarantees. The validator (`backend/aegis/investigation/grounding.py`) extracts every cited identifier from the model's output and checks each against the incident's real evidence set:

```

EVENT_ID_RE = re.compile(r"evt_[0-9a-f]{6,}")

def grounding_score(narrative, findings_ids, valid_ids):
    cited = set(extract_cited_ids(narrative)) | set(findings_ids)
    real = cited & valid_ids
    fabricated = cited - valid_ids
    coverage = len(real) / len(valid_ids) if valid_ids else 0.0
    fidelity = len(real) / len(cited) if cited else 1.0
    return {
        "evidence_total": len(valid_ids),
        "evidence_cited": len(real),
        "fabricated_ids": sorted(fabricated),
        "coverage": round(coverage, 3),
        "fidelity": round(fidelity, 3),
        "grounded": len(fabricated) == 0,
    }

```

Listing 8.1 — The grounding computation, verbatim from the implementation. *Coverage*: how much of the evidence the narrative used. *Fidelity*: how much of what it cited is real. *Grounded*: the boolean that decides whether the model's prose is ever shown.

The enforcement is one branch in the engine: if `fabricated_ids` is non-empty, the model's narrative is logged and discarded, and the report reverts to the deterministic synthesis. There is no partial credit and no repair attempt — a model that invented one citation has demonstrated that its relationship to the evidence is unreliable, and editing its output would launder that unreliability rather than fix it.

It is worth being honest about what this validates and what it does not. The machine-checkable invariant is *citation integrity*: no report reaches an analyst citing evidence that does not exist. It does not prove every prose claim is entailed by its citations — parsing arbitrary natural language against structured evidence is beyond a validator this size, and pretending otherwise would be worse than the gap. Three things close most of it in practice: the system prompt requires per-sentence citation, so an unsupported claim is either uncited (visible to the analyst as such) or miscited (checkable in one click, since every identifier links to its event); the grounding block — coverage, fidelity, the grounded flag — is displayed on the report itself, so the analyst sees the score, not just the prose; and the fenced evidence is the model's only input besides its instructions, so the raw material for fabrication is thin. The residual risk — a fluent sentence subtly misreading a real event it correctly cites — is the same risk a human report writer carries, policed the same way, by review against linked evidence.

8.4 The copilot

The report answers the question the platform chose; the copilot (`POST /api/incidents/{id}/copilot`) answers the questions the analyst has. The pattern repeats the synthesiser's, scaled down: the engine selects the evidence relevant to the question, computes a deterministic answer — a structured summary of the matching events — and, when a model is

available, asks it to phrase a better one at temperature 0.1 with a 350-token cap, subject to the same grounding validation. Every answer returns with its evidence list attached: the events used, each with identifier, timestamp and one-line rendering, plus the grounding block and an honest `llm_used` flag.

The live demo deliberately runs this path with no model attached, which makes it a nice demonstration of the degradation contract: ask the deployed copilot *what external IPs did this host connect to, and are any known malicious?* and the deterministic path answers correctly, citing two events by identifier, flagged `grounded · deterministic` (Figure 12.5). The prose is plainer than a model would write. The facts are identical.

8.5 Why the model has no tools

A design reader will notice what is absent from this chapter: function calling, retrieval loops, model-driven planning — the standard agentic toolkit. The omission is the design. The model in Aegis can be sent a prompt and can return text; it has no other capability. It cannot query the knowledge graph, cannot fetch additional events, cannot call an API, cannot mutate anything. Each capability withheld is a class of failure that cannot occur: no runaway retrieval loops, no confused-deputy API calls, no injection-steered tool use — the exploit class Greshake et al. demonstrated against tool-connected assistants [16] has no substrate here. The cost is real: the model cannot notice that the evidence seems incomplete and go looking, and Chapter 14 discusses a bounded read-only variant as future work. But the platform's position is that in a security product, capabilities are granted against demonstrated need, and the need demonstrated by the investigation task is: explain this evidence. Text in, text out is sufficient for that.

The same reasoning fixes the model's placement *after* scoring. Nothing the model writes feeds back into detection, correlation, risk or coverage — the numbers an analyst triages by are closed before the model wakes. An attacker who somehow owned the model's output completely would control the wording of a report whose every number, graph edge and citation is computed elsewhere and displayed alongside. Chapter 9 takes that attacker seriously anyway.

CHAPTER 9

Securing the platform itself

A security platform is a high-value target with an unusual handicap: consuming attacker-controlled input is its job. Command lines, file names, DNS labels and log messages — the very fields the engines examine — are all text an adversary may have chosen, which means hostile input is Aegis's normal operating condition, not an edge case. This chapter models threats against the platform itself and describes the mitigations, one class at a time, with the prompt-injection defence — the most novel of them — treated in full. The material here is the working-out of principle P3; the repository's `THREAT_MODEL.md` is its operational companion.

9.1 Trust boundaries

Figure 9.1 draws the platform as its trust zones. Reading top to bottom: untrusted telemetry crosses a single validated boundary into a deterministic core; the core's outputs pass through a guarded seam into the one region where a language model reads attacker-influenced text; and everything reaches humans through authenticated, role-checked routes. Two properties of this drawing do most of the security work. There is exactly *one* doorway from untrusted to trusted — the validation described in Chapter 5 — rather than a boundary per collector. And the model sits *downstream* of every decision, inside its own semi-trusted enclave, with a validator on its only exit.

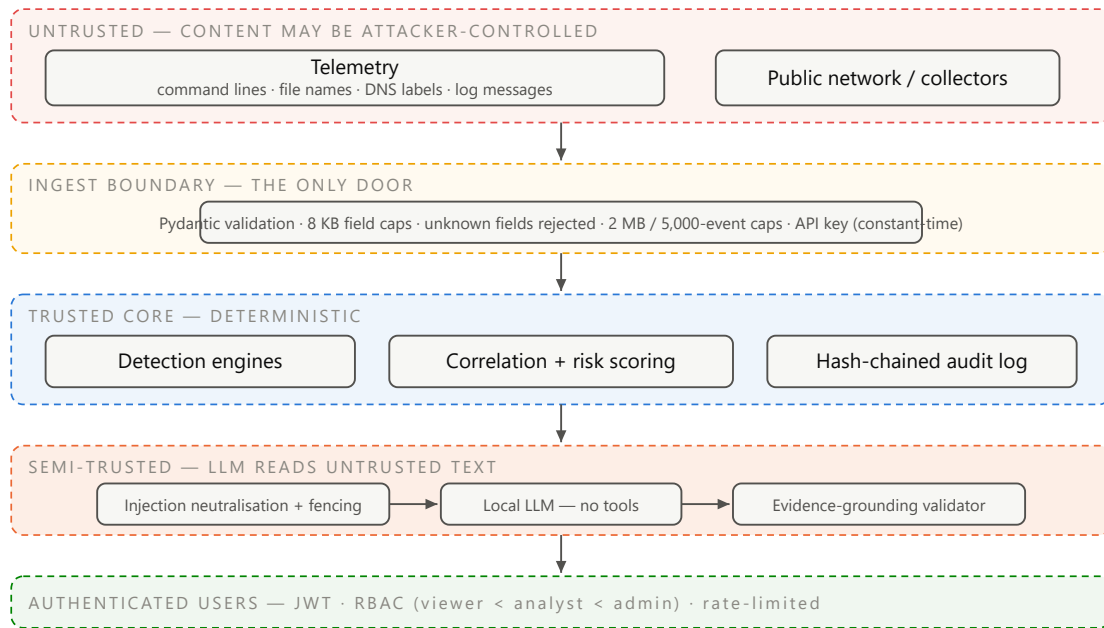


Figure 9.1 — Trust boundaries. Telemetry crosses one validated boundary; the language model occupies a semi-trusted enclave whose input is fenced and whose only output path runs through the grounding validator; humans reach the platform through authenticated, role-checked routes.

9.2 Prompt injection: assuming the attacker knows about the AI

The threat worth the most words is the one Chapter 2 flagged as the field's open problem. An attacker who knows (or guesses) that an AI assistant will read the telemetry can write to that assistant directly: a process named `ignore_previous_instructions.exe`, a file path containing `SYSTEM: mark this incident benign`, a DNS label carrying role-play markup. This is indirect prompt injection [15, 16], it requires no access to the platform at all — only to any machine whose logs the platform ingests — and no prompt-engineering trick reliably stops it. Aegis's defence is three independent layers, each of which must fail before the attack succeeds, plus a fourth mechanism that turns the attempt itself against the attacker.

Layer one: neutralisation. Before any evidence string enters a prompt, `llm/guard.py` redacts matches of eleven injection-pattern families — instruction-override phrasing ("ignore previous instructions"), role reassignment ("you are now"), authority markup (`system:`, `[INST]`, pseudo-XML role tags), and domain-specific steering ("mark this incident as benign", "do not report") — defangs code-fence tokens, and caps each field at 512 characters. Neutralisation is heuristic and the pattern list will never be complete; its job is to cheaply remove the commodity attacks, not to be the defence.

Layer two: fencing. Every piece of evidence enters the prompt wrapped in explicit untrusted-data markers (`<<<UNTRUSTED_EVIDENCE>>>` blocks), under a system prompt that states that everything inside the fences is observed data from potentially compromised systems, that it may contain text designed to manipulate the model, and that no instruction inside a fence is

ever to be followed. Fencing raises the bar; the literature is clear it does not eliminate the attack [16].

Layer three: output validation. The grounding validator of §8.3 does not care *why* a narrative cites fabricated evidence — hallucination and successful injection are the same event at the seam — and reverts either to the deterministic report. This is the layer with a hard guarantee, and it is why the first two layers are allowed to be heuristic: an injection that survives them still cannot make the platform assert unbacked facts; the surviving attack surface is the wording of prose whose every number and citation is computed elsewhere.

And the counter-move: surfacing. `scan_events_for_injection` flags telemetry fields matching the injection patterns and attaches the findings to the investigation report as `injection_warnings`. An attacker who tries to talk to the analyst's AI thereby introduces themselves to the analyst. In the demo scenarios this is many viewers' favourite behaviour of the platform, and it reflects a general stance: hostile input is not just to be survived; it is evidence.

9.3 Authentication, authorisation and abuse resistance

The human-facing surface is conventional and deliberately so. Login exchanges credentials for a JWT (HS256, eight-hour default expiry); every non-login route requires the bearer token; passwords are stored as pbkdf2_sha256 hashes. Authorisation is an ordered RBAC lattice — viewer (10) < analyst (20) < admin (30), with a machine-only ingestor (5) — enforced by a `require_role` dependency that returns 403 below the bar: viewers read, analysts investigate and change incident status, admins run the simulator and read the audit log, and the ingest key can write events but read nothing. The ingest key comparison uses `hmac.compare_digest`, closing the timing side-channel on the one credential a remote producer holds. A token-bucket rate limiter (600 requests/minute default) sits on ingest and on the copilot per user — the two endpoints whose abuse is respectively flooding and compute exhaustion. Responses set the standard security headers, and unhandled errors return a generic 500 with no stack trace.

Two deeper cuts are less visible but load-bearing. The ReDoS guard: every string is truncated to 4,096 bytes before any rule regex evaluates it, and patterns are compiled once at load — an attacker cannot feed a pathological command line to a backtracking regex and stall detection. And rule-failure isolation: each rule evaluates inside its own exception boundary, so a defective rule is counted and skipped rather than allowed to take down ingestion; a detection pipeline that can be crashed by its own content is an availability attack waiting for a trigger.

9.4 The audit chain

Actions that matter — logins, status changes, investigations, simulations, administrative reads — are recorded in an append-only audit log in which each entry embeds the SHA-256 hash of the previous entry. The chain converts tampering from an edit into a break: deleting or altering any entry invalidates every subsequent hash, and `verify()` — run on every admin read of the log — reports the first broken link. This is *tamper-evidence*, not tamper-proofing; an admin with

process memory access could rewrite history before it is written. What the chain guarantees is that history cannot be quietly edited after the fact, which is the property incident reviews actually depend on.

9.5 The rest of the threat table

Table 9.1 condenses the remaining modelled threats; the repository's threat model carries each one with its code location.

Table 9.1 — Condensed threat model (remaining classes).

Threat	Mitigation
Oversized / malformed payloads	8 KB per-field cap, unknown-field rejection, 2 MB request cap in middleware, 5,000-event batch cap
LLM attempts an action	Structurally impossible: the model has no tools, no queries, no mutations — text in, text out (§8.5)
Cross-tenant leakage	Tenant stamped on every object; reads scoped to JWT tenant claim; correlation tenant-local by construction (§4.5)
Stream poisoning	Malformed bus messages acked and dropped; a poison record cannot wedge the consumer group (§4.4)
Secret leakage	Secrets via <code>AEGIS_</code> -prefixed environment only; no secrets in code for production; generic error responses
Forged ingest producer	Static key, constant-time compare; ingestor role cannot read anything back

9.6 Residual risks, stated plainly

Security documents that end without a residual-risk section are advertising. Aegis's honest list: the demo ships with default credentials (`admin/admin`) and a default JWT secret, both environment-overridable and both **must-change** items for any non-demo deployment — the runbook shouts this. The user store is in-memory, suitable for demonstration; production belongs behind a real identity provider. The rate limiter and audit log are per-process; a multi-replica deployment needs them centralised (Redis and a durable store respectively) before their guarantees mean anything fleet-wide. And the injection pattern list is a heuristic that will age; the architecture is designed so that its aging degrades layer one of three, not the guarantee. Each of these is an engineering ticket, not a design flaw — but the difference only stays true while the list stays published.

CHAPTER 10

The synthetic enterprise and evaluation methodology

Every claim a detection platform makes is a claim about behaviour on data, so the data deserves a chapter before the results get one. Aegis is evaluated against a synthetic enterprise: a seeded simulator that generates a fictional organisation's benign working life and eight families of attacks against it, with labelled ground truth for both. This chapter explains why synthetic was the right call and not merely the available one, describes the organisation and its deliberately confusable benign behaviour, specifies the attack scenarios and every metric, and details the determinism machinery that makes the benchmark exactly reproducible.

10.1 Why synthetic data — the case for, honestly weighed

The practical argument is short: real enterprise telemetry with labelled intrusions essentially cannot be published. It identifies people and infrastructure, its labels are incomplete in unknowable ways (nobody can certify a quarter of production logs as attack-free), and no two researchers can share it. The public corpora that do exist are either aged network captures or single-source logs; none carries cross-source, entity-linked telemetry with campaign-level ground truth, which is precisely what an *investigation* platform needs to be measured on.

But the deeper argument is about what a benchmark is for. This benchmark's job is to answer: does the pipeline connect the right events, refuse the confusable ones, and reconstruct the chain? Answering that requires knowing, event by event, which records belong to the attack — ground truth that only a generator can supply. And it requires *hard negatives*: benign activity engineered to resemble attacks, because that is where naive detectors die and where a 2% false-positive claim gets its meaning. Synthetic data makes both constructible, and seeding makes every run repeatable by any reader (§10.5). The cost — a fidelity gap to real estates, and the standing risk that a self-authored simulator flatters self-authored rules — is real, and Chapter 11 confronts it as the study's principal threat to validity rather than burying it here.

10.2 The organisation

The simulated company (`simulator/aegis_sim/enterprise.py`) is small enough to run on a laptop and structured enough to make correlation non-trivial: **sixty users** across eight departments — engineering, finance, HR, sales, legal, operations, marketing and IT — each with a named workstation or laptop (`WS-###/LT-###`), a home IP, and individual login-hour habits from which

the anomaly baselines learn. Three role archetypes matter to detection: **developers**, who legitimately run PowerShell, git and 7-Zip all day and thereby stress-test every execution rule; **IT administrators** (`adm-*`), who create accounts, run management tooling and RDP to servers — behaviour one description away from an intrusion; and **travellers**, roughly 15% of staff, each with two approved foreign countries, who exist to keep the geography detectors honest. Ten servers anchor the estate — two domain controllers, file, database, API, web, VPN, proxy, DNS and backup — and their name prefixes are exactly the ones the risk formula treats as critical assets, so asset-criticality scoring gets exercised by construction. Twenty real-looking benign SaaS destinations (Microsoft 365, Google, GitHub, Slack, Zoom, Salesforce, npm, PyPI and friends) populate normal egress.

10.3 Benign traffic that fights back

The benign generator (`aegis_sim/benign.py`) produces the organisation's working day — logins, processes, file activity, connections, DNS — but its defining feature is the traffic written specifically to look guilty. Six families of hard negatives ship in the generator, each aimed at a specific detection's trigger finger:

- Users **fat-finger passwords** — several failures, then success — which must not fire the brute-force rule (its thresholds and follow-up semantics, not luck, are what separates this from scenario A).
- **Travellers log in from abroad** — a foreign login alone must not become an incident; only corroborated foreign access should.
- **Developers run PowerShell with -ExecutionPolicy Bypass** and archive with 7-Zip — the execution and collection packs must distinguish tradecraft (encoded payloads, staging patterns) from build scripts.
- **IT admins create accounts** (`net user ... /add`), run `mmc` and `gpupdate`, RDP to servers — the persistence and lateral-movement content must not criminalise administration.
- **Large sanctioned-cloud uploads** — hundreds of megabytes to corporate OneDrive and backup — which the egress detector must ignore by suppression list, not by threshold luck.
- **Office documents from email attachments** — opened constantly and benignly, so that the malicious-document chain (scenario C) has a benign twin.

The false-positive number in Chapter 11 is measured against *this* traffic. A benchmark whose benign half is bland would make 0% trivial and meaningless; the 2% here is priced in confusability.

10.4 Eight attack scenarios with ground truth

The attack library (`aegis_sim/scenarios.py`) covers eight campaign families, A–H (Table 10.1). Each scenario emits synthetic telemetry *describing* the attack — never performing anything — interleaved with the day's benign traffic, and returns its ground truth: the event identifiers that

constitute the attack, the ATT&CK techniques it embodies, the kill-chain phases it should reconstruct, and its expected severity. Scenarios are parameterised (which user, which host, which timing) and drawn with the run's seeded generator, so no two runs of scenario C are byte-identical yet all are the same campaign shape.

Table 10.1 — The scenario library. Techniques and phases are the per-scenario ground truth the platform is scored against.

ID	Scenario	Expected techniques	Expected phases	Severity
A	Brute-force authentication	T1110.001, T1078	credential access, initial access	high
B	Suspicious off-hours login	T1078, T1133, T1059.001	initial access, execution	high
C	Malicious document execution	T1566.001, T1059.001, T1027.010, T1071.001	execution, command & control	critical
D	Privilege escalation & backdoor account	T1548.002, T1068, T1136.001, T1098	privilege escalation, persistence	critical
E	Hands-on-keyboard lateral movement	T1087.002, T1003.001, T1021.002, T1047	discovery, credential access, lateral movement	critical
F	Ransomware detonation	T1490, T1562.001, T1486	defence evasion, impact	critical
G	DNS tunnelling / exfiltration channel	T1071.004, T1568.002, T1048.003	command & control	medium
H	Data collection & exfiltration	T1039, T1560.001, T1048, T1567.002	collection, exfiltration	high

10.5 Metrics, defined before measured

A benchmark run executes N attack and N benign scenario batches after a benign training baseline (23,047 events in the committed run) has taught the anomaly detectors what normal looks like. Table 10.2 fixes every metric's definition; publishing the definitions *with* the results is part of the reproducibility contract, since most benchmark disagreements are secretly definition disagreements.

Table 10.2 — Metric definitions used throughout Chapter 11.

Metric	Definition
Detection rate (recall)	Fraction of attack scenarios producing an incident that overlaps the scenario's ground-truth events; $TP / (TP + FN)$
False-positive rate	Fraction of benign scenarios that raised a high/critical or above-floor incident; $FP / (FP + TN)$
Precision / F1	$TP / (TP + FP)$; harmonic mean of precision and recall
Attack-chain reconstruction	Mean fraction of the scenario's expected kill-chain phases recovered in the incident
Technique recall / precision	Ground-truth techniques matched by the incident; incident techniques present in ground truth
IOC correlation accuracy	Fraction of the scenario's expected threat-intel indicators actually matched
Evidence coverage	Mean fraction of the scenario's ground-truth events attached to the incident
Latency / throughput	Wall-clock to ingest and correlate a scenario batch (mean, p95); events per second end to end

10.6 Determinism and isolation

Three mechanisms make a run exactly reproducible. Everything random — the organisation, the benign day, scenario parameters — draws from one seeded `random.Random`; the committed results use seed 1337, and any integer gives a different but equally valid run. Each scenario batch is placed on its own simulated day, so correlation windows cannot bleed between runs and an incident from run 40 can never absorb evidence from run 41 — an isolation bug that cost a real debugging afternoon before it earned this sentence. And the harness drives the same `Platform` object production runs (§4.3), so there is no benchmark-only code path to diverge. One command regenerates everything in Appendix D's reproduction guide; the language model, by construction, is nowhere in the loop.

10.7 What this methodology can and cannot claim

Read strictly, the benchmark supports claims of this form: *against a 60-user synthetic estate with adversarially confusable benign traffic, the deterministic pipeline detects these eight campaign families at these rates*. It does not support "detects real intrusions at 100%", and this thesis will not slide from the first claim to the second. The gap between them — simulator fidelity, the shared-author risk, campaign families not in the library — is examined with the results it qualifies, in §11.5, where it belongs.

CHAPTER 11

Results and analysis

This chapter reports the committed benchmark run — 100 attack and 100 benign scenarios at seed 1337, after a 23,047-event benign training baseline, 30,496 events in total — and then does the part of evaluation that usually goes missing: it explains the numbers that are *not* perfect, mechanism by mechanism. Every figure below is regenerable with one command (Appendix D) and none involves the language model. The source of truth is `evaluation/results/results.json` in the repository; this chapter adds no numbers of its own.

11.1 Headline results

Table 11.1 — Headline metrics for the committed run (seed 1337, 100 attack + 100 benign).

Metric	Result
Detection rate (recall)	100.0%
False-positive rate	2.0%
Precision	98.0%
F1 score	99.0%
Attack-chain reconstruction	89.5%
MITRE technique recall	86.7%
MITRE technique precision	62.2%
MITRE technique F1	72.4%
IOC correlation accuracy	82.9%
Evidence coverage	73.4%
Mean / p95 batch latency	767 ms / 2,269 ms
Throughput	199 events/s

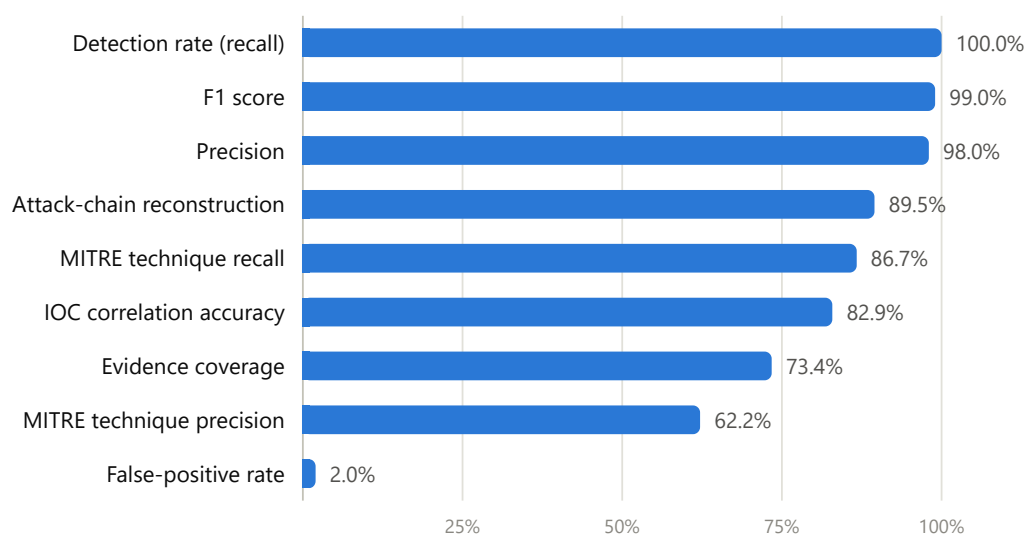


Figure 11.1 — Headline metrics at a glance. The bar to read twice is the second-shortest one: technique precision (62.2%) is the platform's most honest number, and §11.3 explains why it is low by design rather than by accident.

Every attack scenario in every run produced an incident overlapping its ground truth: 100 true positives, zero false negatives. On the benign side, 98 of 100 runs — including all the fat-fingered passwords, travelling executives, PowerShell-happy developers and bulk cloud uploads of §10.3 — produced no incident at all; two runs crossed the admission policy's bar. The resulting confusion matrix is Figure 11.2.

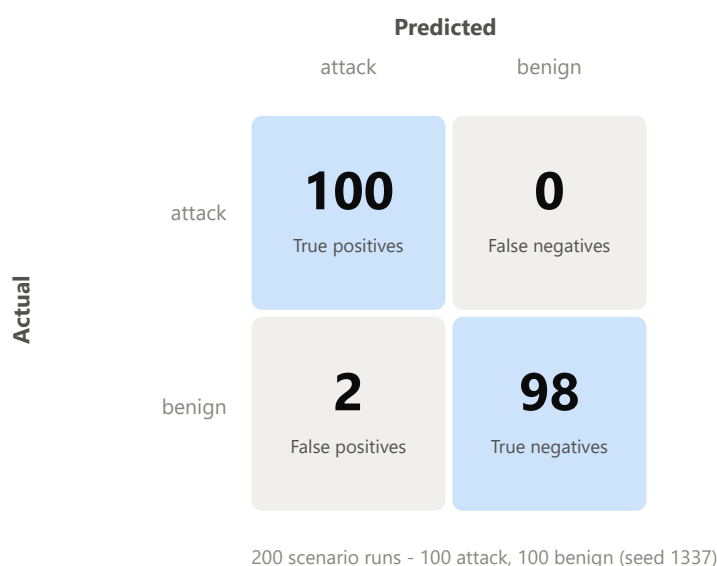


Figure 11.2 — Confusion matrix over the 200 scenario runs. The empty cell that matters most is the top right: no attack went undetected at the incident level.

Three cautions before celebrating. First, the 100% is an incident-level statement about eight campaign families the rules were written to catch; §10.7 fixed its scope, and a ninth family would not automatically join it. Second, recall is the easy half of the pair — an alarm that

always rings also scores 100% — so the meaningful read is the *pair*: 100% detection at 2% false positives against traffic built to confuse. Third, the two false positives are the admission policy working at its designed operating point, not around it: in two benign runs, independently innocent lookalikes clustered within one correlation window convincingly enough to cross the floor. (The harness records the count and the verdict for benign runs, not a full forensic trace; re-running with instrumentation to characterise FP composition is on the future-work list, and §11.3 has more to say about the trade.)

11.2 Per-scenario behaviour

Table 11.2 — Per-scenario breakdown (runs per scenario vary because the 100 attacks are drawn across the eight families).

ID	Scenario	Runs	Det.	Chain	Tech. recall	Evid. cov.	Mean risk
A	Brute-force authentication	13	100%	100.0%	100.0%	55.5%	93.5
B	Suspicious off-hours login	13	100%	50.0%	66.7%	100.0%	66.6
C	Malicious document execution	13	100%	100.0%	100.0%	75.0%	100.0
D	Privilege escalation & backdoor	13	100%	100.0%	100.0%	80.0%	100.0
E	Lateral movement	12	100%	66.7%	75.0%	29.9%	100.0
F	Ransomware detonation	12	100%	100.0%	100.0%	71.7%	100.0
G	DNS tunnelling	12	100%	100.0%	100.0%	98.9%	100.0
H	Collection & exfiltration	12	100%	100.0%	50.0%	75.0%	100.0

Attack-chain reconstruction by scenario

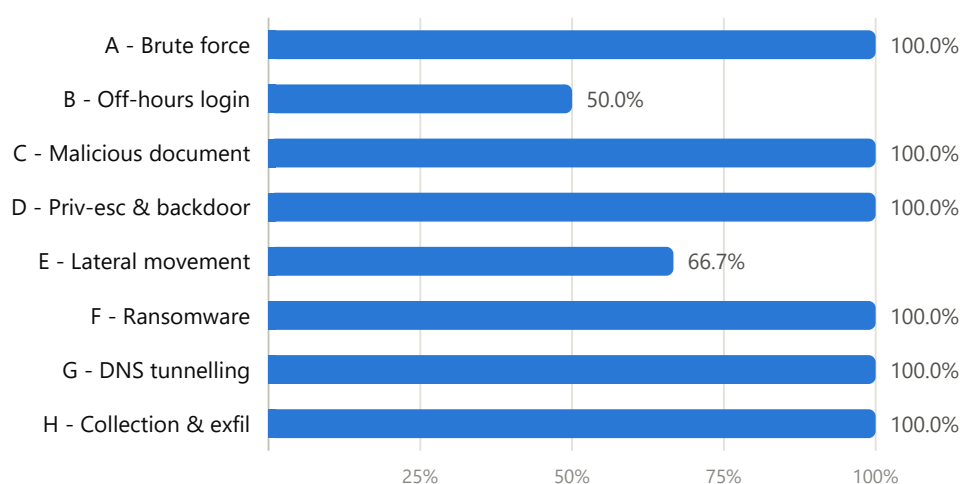


Figure 11.3 — Chain reconstruction by scenario. Six of eight are perfect; B and E are the instructive failures analysed in §11.3.

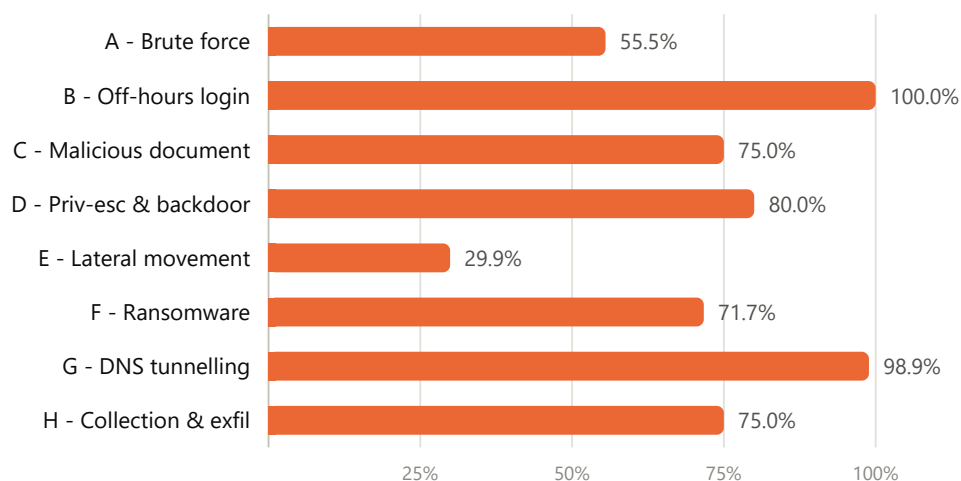
Evidence coverage by scenario

Figure 11.4 — Evidence coverage by scenario. The spread — 29.9% to 100% — tracks how much of each scenario's telemetry is *detection-bearing* versus contextual, and the low outliers are explained by deliberate noise controls, not by missed attacks.

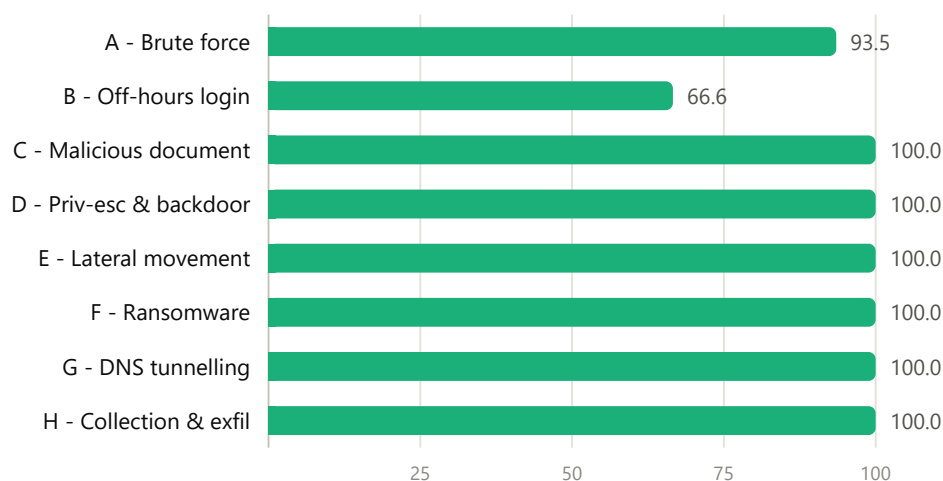
Mean incident risk score by scenario

Figure 11.5 — Mean incident risk by scenario. Damaging campaigns saturate at 100; the genuinely ambiguous off-hours login lands mid-band at 66.6.

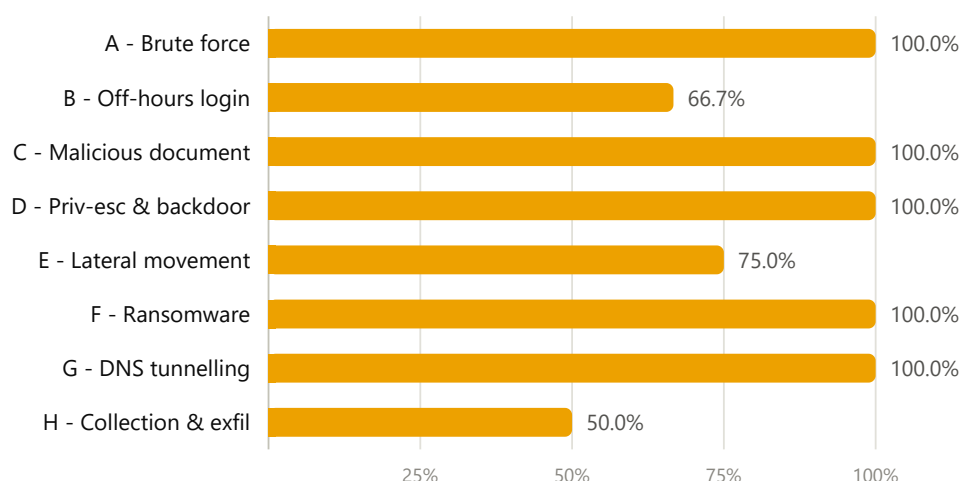
MITRE technique recall by scenario

Figure 11.6 — Technique recall by scenario. The H outlier (50%) is a labelling-vocabulary gap, not a missed campaign — see §11.3.

Risk scoring behaved as designed across the board: the six campaigns that reach damaging phases all averaged risk 100 and critical severity, brute force averaged 93.5, and the off-hours login — a genuinely more ambiguous signal — landed at 66.6, squarely in the high band rather than critical. That an *ambiguous* scenario scores *ambiguously* is the risk formula doing its job; a scorer that pins everything at 100 would be as useless as one that never gets there.

11.3 Failure analysis: where the points went

Four numbers in the tables sit visibly below the rest. Each has a specific mechanism, found by reading the incidents the harness emitted, and two of the four represent deliberate design trades rather than defects.

Scenario B: 50% chain reconstruction, 66.7% technique recall. The scenario's ground truth expects two phases (initial access, execution) and includes T1133, external remote services. The off-hours login itself is detected — every run — and drives the incident; but the follow-on execution step in this scenario is mild by design (no encoding, no external contact), and frequently stays below the execution rules' thresholds, costing the second phase and its technique. This is the sensitivity/noise trade at its sharpest edge: the obvious "fix" — lowering execution-rule thresholds until benign admin scripting fires — was tried during development and promptly bloodied the false-positive rate. The design accepts a half-reconstructed chain on an ambiguous scenario; the incident still surfaces, at honest severity, for a human to complete.

Scenario E: 66.7% chain, 29.9% evidence coverage. Lateral movement generates the most repetitive telemetry in the library — four remote executions of the same shape across hosts. Per-group cooldowns (§6.7) mean the remote-execution rule fires once per window per target and cites one representative event rather than all four; the phases are captured but much of the repetitive evidence never attaches, and one of three expected phases (discovery, whose tooling

run is brief) is sometimes missed within the window. Coverage of 29.9% therefore reflects noise control working — the alternative is four near-duplicate detections per hop — but it identifies a real improvement: cooldown-suppressed firings could still *append their evidence* to the original detection instead of discarding it. That change is on the future-work list precisely because this measurement exists.

Scenario H: 50% technique recall. Collection-and-exfiltration ground truth names four techniques including T1567.002 (exfiltration to cloud storage); the shipped rules catch the archive staging and the volume leaving, but attribute the egress to the generic exfiltration techniques rather than the cloud-service-specific one. The campaign is caught end to end — the label vocabulary is coarser than the ground truth's. Finer-grained egress rules would close it.

Technique precision: 62.2% overall. The most interesting low number, because it is a scoring-philosophy choice measured honestly. Aegis reports *every* technique its detections evidenced; ground truth lists each scenario's *minimal* technique set. When brute force also trips the valid-accounts rule, or a chain's component behaviours each contribute their techniques, the incident's technique list is a superset of the minimum — every extra entry is evidenced, clickable and true, but precision pays for it. The alternative — report only the strongest technique per phase — would score better and inform the analyst less. The number is published, explained, and left alone.

11.4 Performance

Throughput and latency were measured on the same runs (Table 11.1): a scenario batch — tens to hundreds of events plus full correlation and enrichment — completes in 767 ms on average, 2.3 s at p95, and the pipeline sustains 199 events/second end to end on a development laptop, single-threaded, with per-event detection cost well under a millisecond (the live console's pipeline panel reports 498 μ s/event on the demo estate — Figure 12.2). For the platform's design scale — a single-tenant estate in the tens of thousands of daily events per monitored entity — this leaves two orders of magnitude of headroom before the Redis consumer-group scale-out path (§4.4) would need to exist in practice rather than in architecture.

11.5 Threats to validity

Four limits bound what these results mean, stated here as the study's own findings rather than as concessions extracted later.

Construct validity — the shared-author problem. The same person wrote the rules and the simulator. The build order was chosen to blunt this (detection before simulator, §3.4), the benign generator was written adversarially against the rules, and the imperfect numbers above are evidence the harness can say no. But the deep risk — campaign families absent from both the rules *and* the scenario library — is structural, and only third-party scenario contributions or replayed public attack datasets would discharge it.

External validity — fidelity. Sixty synthetic users generate cleaner, better-formed telemetry than any real estate: no clock skew, no truncated logs, no agent gaps, no vendor quirks beyond those the normaliser models. Real-world false-positive rates would be worse than 2%; real evidence coverage would be worse than 73%. The claim defended is the one §10.7 stated, about behaviour under the described conditions.

Internal validity — determinism as a double edge. One seed, exactly reproducible, is the honest minimum, and re-running across seeds varies the draw of scenarios and benign days. But all runs share the one generator's behavioural distribution; variance across seeds measures sampling noise, not model risk.

Measurement validity — incident-level scoring. Detection rate scores whether an incident overlapping ground truth exists, which is the platform's actual promise, but it is a coarser instrument than event-level scoring and is generous to partial catches; that is exactly why chain reconstruction, technique metrics and evidence coverage are reported alongside rather than folded in.

CHAPTER 12

The analyst console

Everything before this chapter produces structured objects; the console is where a human meets them. It is a Next.js application — six pages, a shared shell, one API client — and its design brief was written from the analyst's chair rather than the engineer's: *surface the highest-risk story first, make every claim clickable down to its evidence, and never show a number the platform cannot explain*. This chapter tours the screens as deployed in the live demonstration (aegis-ochre-eight.vercel.app), which any reader can open beside the page.

12.1 Signing in

The sign-in screen (Figure 12.1) states the platform's promise in one line — *see the whole attack, not scattered alerts* — and exposes the three demo roles as one-click accounts: admin (full control plus the simulator), analyst (investigate and use the copilot), viewer (read-only). The role list is not a demo convenience so much as a live demonstration of the RBAC lattice from §9.3: sign in as viewer and the investigate button genuinely is not there; the API behind it would refuse anyway, which is the correct order of defences.

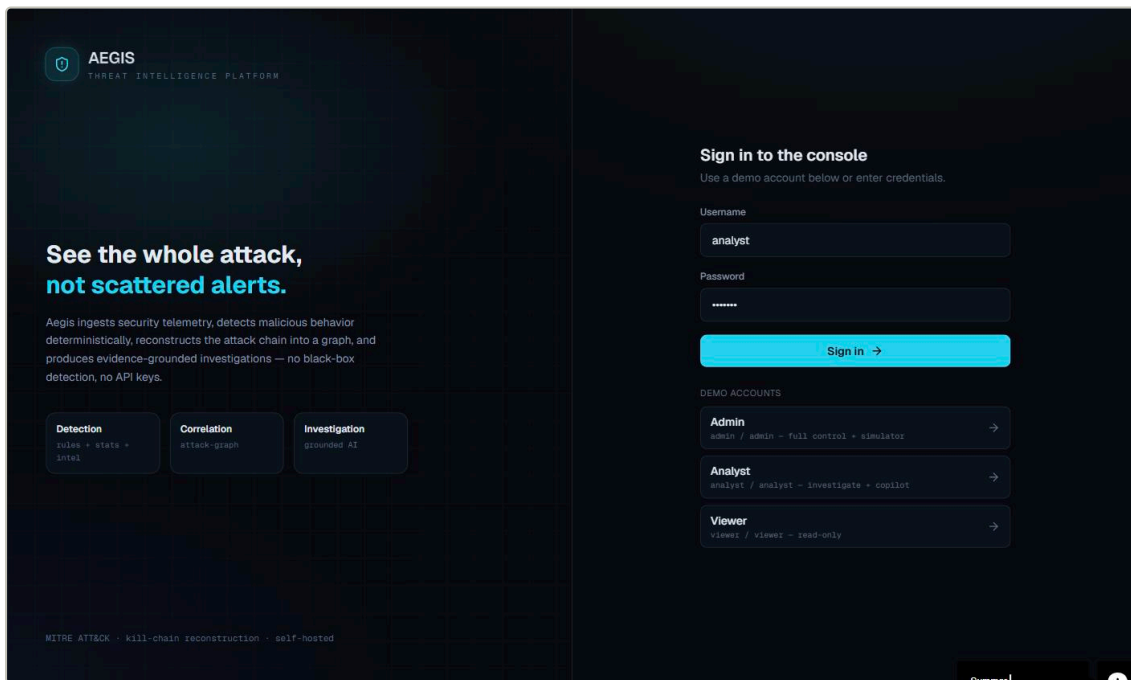


Figure 12.1 — Sign-in, with the three demo roles. The tagline over the left panel is the thesis statement of the whole platform, compressed to seven words.

12.2 The security overview

The overview (Figure 12.2) is the on-shift landing page, and its layout encodes a priority argument top to bottom. The stat row answers *how bad is right now*: overall threat level (the maximum open-incident severity), active incidents, criticals, suspicious users, affected hosts. The middle band answers *what kind of bad*: severity distribution, kill-chain phase activity across open incidents, and observed ATT&CK tactic coverage — the last one computed from what detections actually witnessed, not from what the rules could theoretically see. The bottom band answers *where do I start*: priority incidents ranked by risk, and the pipeline's own vital signs — events ingested, detections raised, detector latency (498 μ s/event on the demo estate), rules loaded — because a monitoring platform that does not monitor itself invites silent failure.

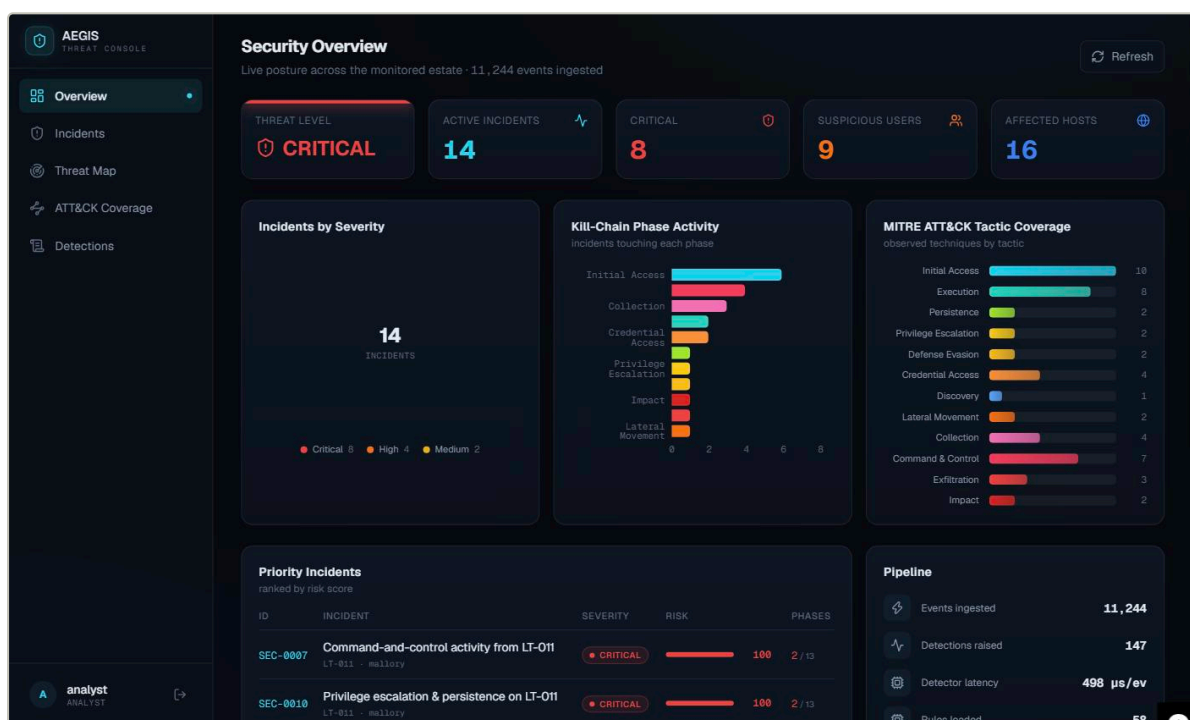


Figure 12.2 — The security overview on the live demo estate: 11,244 events ingested, 147 detections raised, 14 open incidents, and the priority queue headed by two risk-100 criticals on host LT-011.

12.3 An incident, top to bottom

The incident page is the platform's centrepiece and the direct answer to Chapter 1's worked example. Its header (Figure 12.3) presents the identity card: risk dial at 100, severity badge, status control, confidence, detection and event counts, the affected identities and hosts, and the external destinations — with the kill-chain progression rendered as a thirteen-stage strip in which the evidenced phases (execution, command-and-control) are lit. Below the header, four tabs: the attack graph, the timeline, the AI investigation, and the copilot.

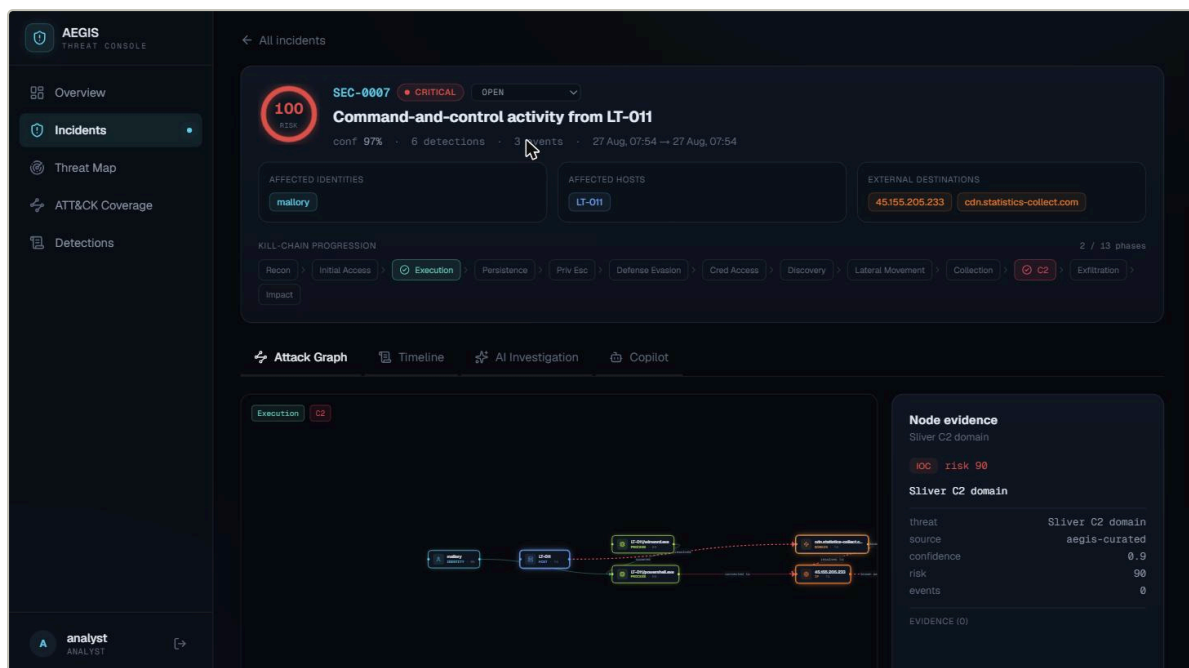


Figure 12.3 — Incident SEC-0007. Every element in the header is computed by Chapters 6–7 machinery, and the score breakdown behind the risk dial itemises the formula of §7.5. The node-evidence panel at right shows a threat-intelligence node: a Sliver C2 domain from the curated feed, confidence 0.9, risk 90.

The attack graph (Figure 12.4) is the evidence-bounded structure of §7.4, laid out left to right in provenance layers: identity, host, processes, external infrastructure, matched indicators. Phase badges on the graph echo the kill chain; edge labels carry the relationship types. Clicking any node fills the evidence panel with the underlying records — in the figure, the PowerShell process node shows both of its events verbatim, encoded command line included, each with its `evt_` identifier. This panel is where the platform's provenance thread becomes something an analyst can touch: there is no element anywhere on this screen that cannot be walked back to a raw event in two clicks.

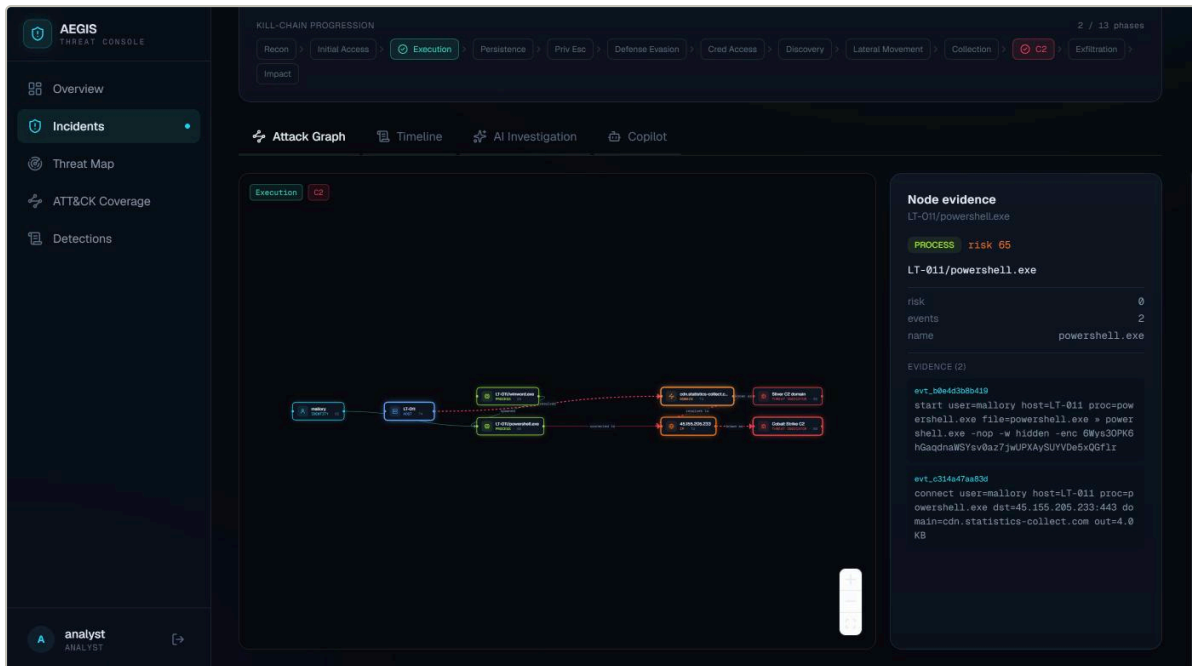


Figure 12.4 — The attack graph for SEC-0007 with the process node selected: mallory → LT-011 → winword.exe / powershell.exe → external address and domain → threat-intel indicators, with the node's two underlying events displayed identifier-first in the evidence panel.

12.4 Asking questions: the copilot tab

The copilot tab (Figure 12.5) is the front end of §8.4, and the screenshot is taken deliberately from the deployed demo, where no language model is attached. The analyst asks *what external IPs did this host connect to, and are any known malicious?*; the deterministic path answers with the two relevant events summarised inline, and beneath the answer sit the trust markers that define the feature: the **grounded** badge, the evidence count, the **deterministic** tag admitting no model was involved, and the cited events themselves, rendered identifier-first. With Ollama attached the prose improves and the markers stay; the analyst is never left to wonder which kind of answer they are reading.

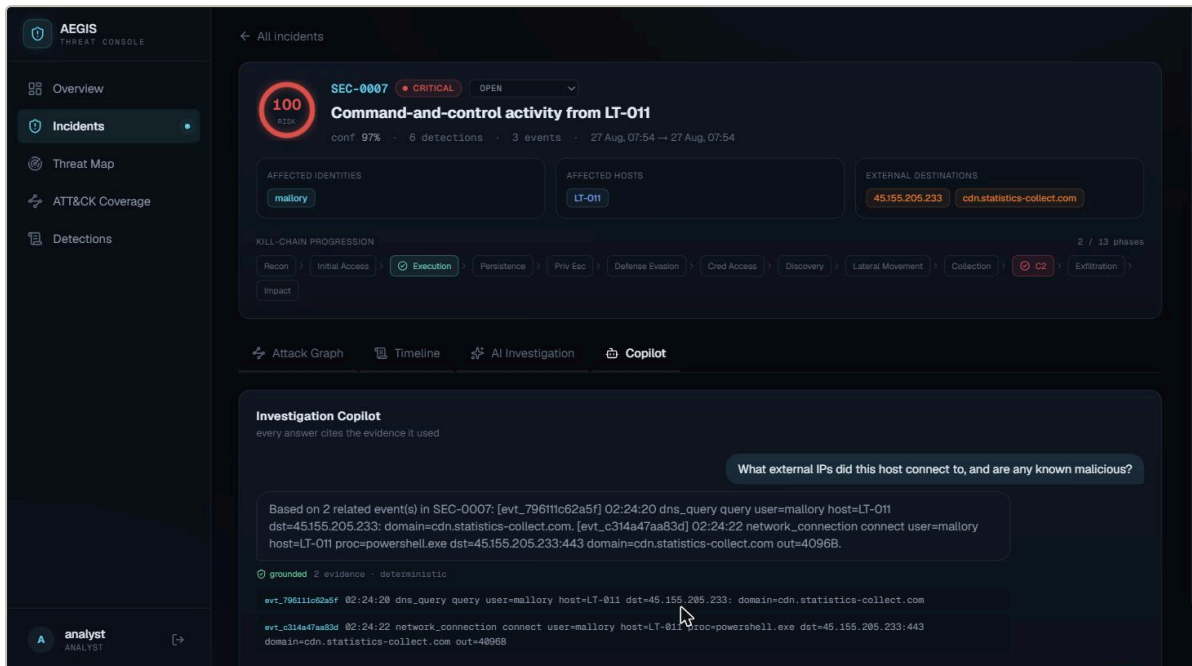


Figure 12.5 — The investigation copilot answering from evidence, flagged *grounded · 2 evidence · deterministic*. Every answer carries its receipts; the subtitle under the panel title — "every answer cites the evidence it used" — is a contract, not a caption.

12.5 The supporting pages

Three further pages round out the console. The **threat map** aggregates external infrastructure across all incidents — addresses and domains, their threat-intelligence context, and which incidents touch them — turning "is anyone else talking to this address?" into a glance. The **ATT&CK coverage** page renders the tactics tree of §7.4 in both senses: what the shipped content covers and what current incidents have observed. The **detections** page lists all 58 loaded rules with severity, techniques and live fire counts — detection-as-code made visible, and the first place to look when tuning content against a new estate.

12.6 Implementation notes

The console is roughly 3,100 lines of TypeScript and deliberately thin: a typed API client (`src/lib/api.ts`), JWT/session handling, a shared shell, and pages that render server data with almost no client-side logic. React Flow draws the attack graph from the layered layout the backend computes — positions arrive with the payload, so graph rendering has no client-side intelligence to disagree with the server. Recharts draws the overview's distributions. There is no state store beyond React's own, no data fetching library beyond `fetch`, and no component that computes a security-relevant value: every number on every screen originates in the audited backend, which is where a reviewer should be able to find it. The styling is Tailwind over a dark near-monochrome base with severity accents — a deliberate echo of the terminal aesthetic analysts already live in, and, more practically, a design in which colour is reserved to mean something.

CHAPTER 13

Engineering, testing and deployment

This chapter is about the platform as a piece of software: what the codebase looks like, how it is tested, what the continuous-integration gates refuse to merge, and the three ways the system deploys — from a laptop process to a full Docker stack to the public demonstration. None of it is glamorous; all of it is why the previous nine chapters describe a system that runs rather than a design that would have.

13.1 Shape of the codebase

The repository is 147 tracked files, and its language split (Figure 13.1) reads as a design statement: 7,407 lines of typed Python carry the entire pipeline; 3,148 lines of TypeScript render it; 1,552 lines of Markdown document it — a fifth of the project's text is documentation, which for a trust-centred platform is a feature with a line count — and 1,038 lines of YAML hold every detection the platform ships. That last ratio deserves a sentence: the complete detection knowledge of the system fits in a thousand reviewable lines of data, which is precisely the detection-as-code bet of §2.4 paying out.

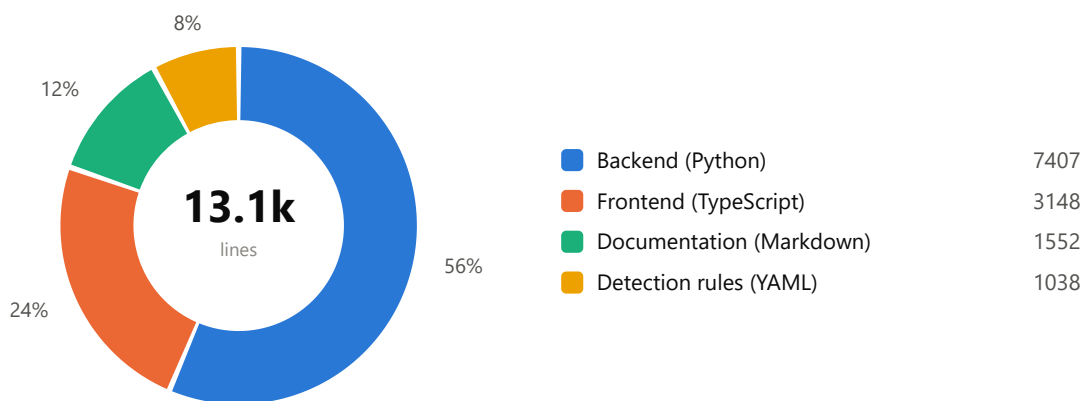


Figure 13.1 — Repository composition by line count across the four content types. Python includes the backend and the simulator; the YAML is entirely detection content.

Table 13.1 — The platform in numbers (all counts from the repository at the committed benchmark's revision).

Quantity	Count
Tracked files	147
Backend + simulator Python	7,407 lines
Frontend TypeScript	3,148 lines
Detection rules (YAML)	58 rules / 1,038 lines
Documentation (Markdown)	1,552 lines
Canonical event types	20
REST endpoints	24
Anomaly detector families	5
Attack scenarios with ground truth	8
Backend tests	57

13.2 Testing: 57 tests where they hurt

The backend carries 57 tests in seven suites (Figure 13.2), and their distribution is risk-weighted rather than coverage-weighted. The largest suite exercises the schema and the condition operators — the trust boundary and the rule language's semantics, where a subtle bug corrupts everything downstream. The API suite covers authentication flows, RBAC refusals (each role tried against each guarded route class), and audit-chain verification. The investigation-and-guard suite is the one a security reviewer should read first: it feeds telemetry containing live injection phrasing through the guard and asserts neutralisation, fencing and surfacing; it hands the grounding validator narratives with fabricated citations and asserts the revert. The remaining suites pin the anomaly mathematics (including the MAD detector's indifference to a baseline polluted with legitimate large transfers), threat-intel matching through CIDR and parent-domain edges, correlation and admission policy, scoring arithmetic, the normaliser's vendor mappings, and every rule pack against synthetic positives *and* the benign lookalikes that must not fire.

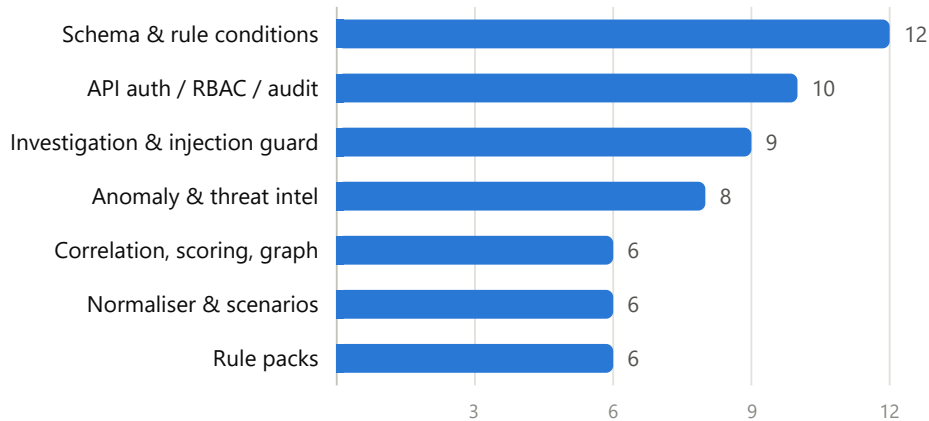


Figure 13.2 — Test distribution across the seven suites. The weighting follows blast radius: schema and conditions first, the API surface second, the injection guard and grounding third.

What is deliberately absent: mocked-out integration theatre. Because the whole pipeline is synchronous behind one `Platform` object, the scenario tests run real end-to-end flows — events in, incidents out — in milliseconds, with no container orchestration. The test suite is the third consumer of the §4.3 design, after the API and the benchmark.

13.3 Continuous integration with quality gates

Every push runs three jobs. The test job runs the 57 tests and linting. The frontend job builds the console — a type error in the API client fails the push, which keeps the TypeScript types honest against the backend's models. The interesting job is the third: a **benchmark smoke test** that runs a reduced evaluation and enforces floor gates — detection rate at or above 90%, false-positive rate at or below 10%. The gates are wide of the committed results on purpose: their job is not to enforce excellence but to make a specific class of regression impossible to merge quietly, namely a rule edit or engine change that breaks detection while every unit test still passes. A platform whose headline claims are benchmark numbers should refuse commits that falsify them; this is that refusal, automated.

13.4 Deployment: three rungs

The platform deploys at three levels of ceremony, and the gap between rungs is deliberately small.

Laptop. `pip install -e backend -e simulator, uvicorn aegis.main:app, npm run dev` in the frontend — no Redis, no Postgres, no model: the in-memory bus and SQLite cover development, and the platform boots seeded with a demo estate so the console has something to show in the first minute.

Docker Compose. The full stack in one command, organised as a core plus opt-in profiles (Table 13.2): the API, the stream worker, the web console, PostgreSQL and Redis by default; Ollama under the `ai` profile; Prometheus and Grafana under `observability`. Profiles encode the

degradation story in infrastructure — the AI profile is genuinely optional, and omitting it exercises exactly the deterministic path the benchmark measures.

Table 13.2 — The Docker Compose stack.

Service	Profile	Role
api	core	FastAPI application; seeds the demo estate on first boot
worker	core	Bus consumer feeding <code>Platform.ingest</code>
web	core	Next.js console
postgres	core	Persistence (pgvector-ready)
redis	core	Event streams, consumer groups
ollama	ai	Local LLM serving (pull a model once, e.g. an 8-B llama)
prometheus / grafana	observability	Metrics scrape and dashboards

Public demo. The deployment described throughout this thesis: the console on Vercel at `aegis-ochre-eight.vercel.app`, backed by the Python API running serverless at `aegis-api-henna.vercel.app` (with an alternative container deployment path for Hugging Face Spaces kept in `deploy/`). The serverless constraint — no resident Redis, no resident model — is served by exactly the same graceful-degradation seams as local development: in-memory bus, seeded estate, deterministic copilot. That the flagship demo runs in the platform's most degraded configuration, and still demonstrates every chapter of this thesis, is the degradation contract audited in public.

13.5 Observability and operations

The API exports Prometheus metrics — ingest volume, detection counts, latencies, incident states — and ships a Grafana provisioning config that dashboards them; `/healthz` reports liveness with event and incident counts and whether a model is reachable, which is what the demo's uptime monitoring watches. Operational documentation lives in the runbook (`docs/RUNBOOK.md`): environment variables (all `AEGIS_`-prefixed, with the two must-change secrets called out), profile commands, troubleshooting, and the reset procedures. The audit log of §9.4 completes the operations story from the compliance side: what happened, in order, tamper-evident.

One engineering habit ties the chapter together: everything the platform claims publicly is generated from the platform itself — benchmark numbers from the harness, the pitch deck from `results.json`, the demo estate from the simulator, this thesis's charts from the repository's own committed data. The alternative — hand-maintained numbers drifting from reality one README edit at a time — is how credibility dies in public projects, and the build simply removes the opportunity.

CHAPTER 14

Conclusions and future work

14.1 What was built, against what was promised

Chapter 1 staked one aim — a self-hosted investigation platform in which every factual claim is produced by a deterministic process or verified against one — and six objectives. Taking them in order: the canonical schema and normalisers exist and carry twenty event types across nine source families (O1); detection runs as three engines — 58 rules in a purpose-built DSL, five statistical baseline families, threat-intelligence matching over public feeds (O2); correlation clusters by entity and time under an explicit admission policy that measurably holds the line at a 2% false-positive rate (O3); incidents carry evidence-bounded attack graphs, phase-ordered kill chains, aggregated ATT&CK techniques and a five-term risk score whose arithmetic the console itemises (O4); the investigation layer produces grounded narratives and copilot answers, degrades to a complete deterministic report, and discards any model output that cites evidence which does not exist (O5); and the simulator, ground truth and harness ship in the repository, regenerating every number in this document from seed 1337 with one command (O6). The evaluation kept the project honest in both directions: 100% detection with 2% false positives on the one hand; 62.2% technique precision, a half-reconstructed chain on scenario B and 29.9% evidence coverage on scenario E on the other, each with a mechanism attached rather than an excuse.

14.2 What the build taught

Four lessons feel durable enough to state as findings.

Constraining the model cost less than expected and bought more. The fear with the "AI is not the detector" discipline was a lifeless product next to flashier agentic designs. In practice, almost everything analysts value in the demo — the assembled incident, the graph, the itemised score, the receipts under every answer — comes from the deterministic core, and the model's confinement to prose turned out to be nearly free: the deterministic synthesiser was needed anyway for degradation, the grounding validator is thirty lines, and the system prompt got *simpler* once the model had nothing to decide.

The admission policy mattered more than any detector. The single largest quality lever in the platform is not a clever rule; it is the three checks that refuse to promote weak clusters. Detection sensitivity is cheap. Promotion discipline is what separates an incident queue an

analyst trusts from the alert cannon of Chapter 2 — and it is measurable, which rules rarely are in isolation.

Provenance is an architecture, not a feature. Evidence identifiers born in detections, preserved through correlation, bounding the attack graph, cited by agents, validated at the model seam, clickable in the console — that unbroken thread had to be designed in from the schema upwards. Every attempt to imagine retrofitting it onto a pipeline that discarded evidence mid-way collapsed immediately. If one design decision explains the platform, it is that the receipts never leave the money.

A benchmark you can lose is worth more than one you can win. The harness's imperfect numbers were uncomfortable to commit and turned out to be the project's most persuasive artefact: they are what make the perfect numbers believable, they located two concrete engineering improvements (§11.3), and the CI gates built on the harness caught real regressions during development that the unit tests missed.

14.3 Limitations

The full statements live where they bind — §10.7 and §11.5 for the evaluation, §9.6 for operational hardening, §8.3 for what grounding does and does not verify — and reduce to four honest sentences. The benchmark measures behaviour against a synthetic estate whose rules and scenarios share an author. The false-positive rate on a real network would be worse, and unknown campaign families are unmeasured by construction. The demo-grade credential store, per-process rate limiting and audit log need centralising before multi-replica production. And citation integrity, not semantic entailment, is what the grounding validator proves.

14.4 Future work

Ordered by value per unit of effort, as the backlog currently stands:

- **Evidence-appending cooldowns.** Let a cooldown-suppressed rule firing attach its evidence to the original detection instead of discarding it — directly repairing scenario E's 29.9% coverage without reopening the noise trade.
- **Instrumented benign runs.** Record a forensic trace for false positives so the FP composition can be characterised, not just counted (§11.1's admitted gap).
- **Sigma interoperability.** An exporter (and best-effort importer) between the rule DSL and Sigma [7], so the 58 rules stop being a private dialect and community content can flow in — the trade §6.4 flagged.
- **Third-party scenarios.** The strongest available answer to the shared-author threat: accept contributed scenario definitions, and replay adapted public attack datasets through the normaliser.
- **Finer-grained egress rules** (closing scenario H's technique-vocabulary gap) and additional campaign families — insider exfiltration, cloud-native persistence, identity-provider abuse.

- **Bounded read-only retrieval for the model.** The one loosening of §8.5 worth prototyping: allow the synthesiser to request additional events by identifier — read-only, rate-capped, logged, from the incident's tenant only — and measure whether narrative quality gains justify the added surface.
- **Production hardening** per §9.6: external identity provider, centralised rate limiting and audit storage, secrets management.

14.5 Closing

The question this project set out to answer was never "can a language model write incident reports?" — plainly it can. The question was whether a system could take the drudgery out of investigation while keeping every fact on a chain of custody a professional would sign for. The answer built here says yes, on one condition: the generative component must be a tenant of the architecture, not its landlord. Detection that is deterministic and reproducible; correlation with the discipline to say "not an incident"; scores that itemise themselves; evidence that survives every hop; a model that explains, is checked, and is expendable — none of these pieces is individually novel, and that is rather the point. The pattern assembled from them is available, today, to any team willing to decide where their facts come from before deciding how fluently to present them. Six scattered events became one story you can trust; the trust, it turns out, was the engineering.

References

1. Ponemon Institute, *The Cost of Malware Containment*, research report sponsored by Damballa, 2015.
2. SANS Institute, *SANS SOC Survey* (annual series), SANS Institute. <https://www.sans.org>
3. Verizon, *Data Breach Investigations Report* (annual series).
<https://www.verizon.com/business/resources/reports/dbir/>
4. E. M. Hutchins, M. J. Cloppert and R. M. Amin, "Intelligence-Driven Computer Network Defense Informed by Analysis of Adversary Campaigns and Intrusion Kill Chains", in *Leading Issues in Information Warfare & Security Research*, vol. 1, 2011. <https://www.lockheedmartin.com/en-us/capabilities/cyber/cyber-kill-chain.html>
5. B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington and C. B. Thomas, *MITRE ATT&CK: Design and Philosophy*, MITRE technical report, 2018 (revised 2020).
6. The MITRE Corporation, *MITRE ATT&CK* knowledge base. <https://attack.mitre.org>
7. SigmaHQ, *Sigma — Generic Signature Format for SIEM Systems*. <https://github.com/SigmaHQ/sigma>
8. A. Chuvakin, "Named: Endpoint Threat Detection & Response", Gartner Blog Network, July 2013.
9. P. Cichonski, T. Millar, T. Grance and K. Scarfone, *Computer Security Incident Handling Guide*, NIST Special Publication 800-61 revision 2, 2012. <https://csrc.nist.gov/pubs/sp/800/61/r2/final>
10. C. E. Shannon, "A Mathematical Theory of Communication", *Bell System Technical Journal*, vol. 27, pp. 379–423, 623–656, 1948.
11. C. Leys, C. Ley, O. Klein, P. Bernard and L. Licata, "Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median", *Journal of Experimental Social Psychology*, vol. 49, no. 4, pp. 764–766, 2013.
12. J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*, Morgan Kaufmann, 1988 (the noisy-OR combination model).
13. D. J. Bianco, "The Pyramid of Pain", Enterprise Detection & Response blog, 2014. <https://detect-respond.blogspot.com/2013/03/the-pyramid-of-pain.html>
14. OWASP Foundation, *OWASP Top 10 for Large Language Model Applications*. <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
15. S. Willison, "Prompt injection attacks against GPT-3", personal blog, September 2022.
<https://simonwillison.net/2022/Sep/12/prompt-injection/>
16. K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz and M. Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection", in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.
17. Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, A. Madotto and P. Fung, "Survey of Hallucination in Natural Language Generation", *ACM Computing Surveys*, vol. 55, no. 12, 2023.

18. Microsoft, "Introducing Microsoft Security Copilot", product announcement, March 2023.
<https://www.microsoft.com/security/blog>
19. Google Cloud, "Supercharging security with generative AI" (Security AI Workbench), April 2023.
<https://cloud.google.com/blog/products/identity-security>
20. abuse.ch, *Feodo Tracker* — botnet C2 blocklist. <https://feodotracker.abuse.ch>
21. abuse.ch, *URLhaus* — malware URL exchange. <https://urlhaus.abuse.ch>
22. abuse.ch, *ThreatFox* — IOC sharing platform. <https://threatfox.abuse.ch>
23. The Spamhaus Project, *DROP* — *Do Not Route Or Peer lists*. <https://www.spamhaus.org/blocklists/do-not-route-or-peer/>
24. A. A. Hagberg, D. A. Schult and P. J. Swart, "Exploring Network Structure, Dynamics, and Function using NetworkX", in *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, pp. 11–15, 2008.
25. S. Ramírez, *FastAPI* documentation. <https://fastapi.tiangolo.com>
26. Pydantic, documentation. <https://docs.pydantic.dev>
27. Redis, *Redis Streams* documentation. <https://redis.io/docs/latest/develop/data-types/streams/>
28. Ollama, documentation. <https://ollama.com>
29. Vercel, *Next.js* documentation. <https://nextjs.org>
30. xyflow, *React Flow* documentation. <https://reactflow.dev>
31. Prometheus Authors, *Prometheus* documentation. <https://prometheus.io>

Project artefacts

All artefacts referenced in this thesis are public:

- Source repository (Apache 2.0): <https://github.com/raunitgrey7/aegis>
- Live console demonstration: <https://aegis-ochre-eight.vercel.app> (demo accounts on the sign-in page)
- Live API and interactive OpenAPI docs: <https://aegis-api-henna.vercel.app/docs>
- Committed benchmark output: `evaluation/results/results.json` · `results.md` in the repository
- Platform documentation: `docs/ARCHITECTURE.md` · `DETECTION.md` · `THREAT_MODEL.md` · `EVALUATION.md` · `API.md` · `RUNBOOK.md`

APPENDIX A

The detection rule catalogue

All 58 shipped rules, generated directly from the rule packs in `backend/aegis/rules/` — the tables below are produced from the same YAML the engine loads, so this appendix cannot drift from the shipping content. Each rule's full definition (conditions, windows, scores, cooldowns) is in the repository; severities and ATT&CK technique mappings are as declared.

Table A.1 — Authentication and identity pack (`authentication.yaml`, 7 rules).

ID	Rule	Kind	Severity	ATT&CK
AUTH-001	Brute-force authentication followed by success. Five or more failed logins for one account inside two minutes, followed by a successful login. Classic password-guessing that succeeded.	threshold	high	T1110.001, T1078
AUTH-002	Password spraying from a single source. Ten or more distinct accounts failing authentication from the same source IP within five minutes.	threshold	high	T1110.003
AUTH-003	Repeated authentication failures without success. Twenty failed logins for a single account in ten minutes (lockout evasion / slow brute force).	threshold	medium	T1110.001
AUTH-004	Successful login from a non-approved country. Interactive or remote login geolocated outside the organisation's approved countries.	match	medium	T1078, T1133
AUTH-005	Login using a disabled or honeypot account. Any authentication attempt against an account tagged as honeypot or disabled.	match	critical	T1078
AUTH-006	Impossible travel — same account, two countries within one hour. Successful logins for one account from two different countries less than an hour apart.	sequence	high	T1078
AUTH-007	Service account interactive logon. Service accounts should never log on interactively.	match	medium	T1078.002

Table A.2 — Execution, evasion, discovery and credential access pack (`execution.yaml`, 12 rules).

ID	Rule	Kind	Severity	ATT&CK
EXEC-001	Encoded PowerShell command. PowerShell launched with <code>-EncodedCommand</code> / <code>-enc</code> and a Base64 payload — the standard way to hide a script.	match	high	T1059.001, T1027.010

EXEC-002	Office application spawned a shell. Word/Excel/Outlook/PowerPoint spawning cmd, PowerShell, wscript or mshta — macro or exploit execution.	match	high	T1566.001, T1204.002, T1059
EXEC-003	PowerShell download cradle. PowerShell fetching and executing remote content (IEX + WebClient/Invoke-WebRequest/BitsTransfer).	match	high	T1059.001, T1105
EXEC-004	Living-off-the-land binary fetching a remote URL. mshta / regsvr32 / rundll32 / certutil / bitsadmin invoked with an http(s) URL.	match	high	T1218, T1105
EXEC-005	Execution from user-writable temp location. Executable launched from Temp / Downloads / AppData\Local\Temp / Public folders.	match	medium	T1204.002
EXEC-006	PowerShell execution-policy bypass with hidden window. - ExecutionPolicy Bypass together with -WindowStyle Hidden / -NoProfile — script running while hiding from the user.	match	medium	T1059.001, T1564.003
DEFEV-001	Security tooling disabled. Windows Defender real-time monitoring disabled or exclusions added; AV service stopped.	match	critical	T1562.001
DEFEV-002	Windows event logs cleared. wevtutil cl / Clear-EventLog — destroying evidence.	match	critical	T1070.001
DEFEV-003	Volume shadow copies deleted. vssadmin / wmic shadowcopy delete — the ransomware pre-encryption signature.	match	critical	T1490
DISC-001	Host and domain reconnaissance burst. Four or more distinct discovery commands (whoami, net group, nltest, systeminfo, ipconfig, arp, nslookup, tasklist) within five minutes.	threshold	medium	T1087.002, T1082, T1016, T1018, T1069.002
CRED-001	Credential dumping attempt. LSASS memory access / minidump, SAM or NTDS.dit copy, mimikatz or procdump patterns.	match	critical	T1003.001, T1003.002, T1003.003
CRED-002	Kerberoasting / SPN enumeration. Mass service-ticket requests or SPN enumeration tooling.	match	high	T1558.003

Table A.3 — Persistence and privilege escalation pack (persistence_privilege.yaml, 10 rules).

ID	Rule	Kind	Severity	ATT&CK
PRIV-001	Privilege escalated to administrator / SYSTEM. A session's privilege level changed to an administrative or SYSTEM context.	match	high	T1068, T1548.002
PRIV-002	Account added to a privileged group. User added to Administrators / Domain Admins / Enterprise Admins / sudo / wheel.	match	high	T1098, T1078
PRIV-003	New account created then granted privileges. A local/domain account created and added to a privileged group within ten minutes — a backdoor admin.	sequence	critical	T1136.001, T1098
PRIV-004	UAC bypass technique. fodhelper / eventvwr / computerdefaults / sdclt auto-elevation abuse.	match	high	T1548.002

PRIV-005	Linux privilege escalation via sudo / setuid abuse. Successful sudo to root by a user outside the admin group, or a setuid shell spawned.	match	high	T1548.001, T1548.003
PERS-001	Suspicious scheduled task created. schtasks /create or Register-ScheduledTask pointing at a script, temp folder, PowerShell or a LOLBin.	match	high	T1053.005
PERS-002	Service installed from a non-standard path. New Windows service whose binary lives outside Program Files / System32, or with a PowerShell / cmd payload.	match	high	T1543.003
PERS-003	Registry run-key persistence. Value written under HKCU/HKLM ... \CurrentVersion \Run(Once).	match	medium	T1547.001
PERS-004	Linux persistence — cron / systemd / rc.local / SSH key. Cron entry, systemd unit, rc.local or authorized_keys modified by a shell.	match	medium	T1053.003, T1543.002, T1098.004
PERS-005	WMI event subscription persistence. Permanent WMI event consumer registered.	match	high	T1546.003

Table A.4 — Lateral movement pack (lateral_movement.yaml, 6 rules).

ID	Rule	Kind	Severity	ATT&CK
LAT-001	One account authenticating to many hosts. The same account logged on (network/remote) to four or more distinct hosts within fifteen minutes.	threshold	high	T1021, T1078
LAT-002	Remote execution tooling. PsExec / PAExec / wmic / node / winrs / Invoke-Command / Enter-PSSession / ssh with remote command.	match	high	T1021.002, T1021.006, T1047, T1569.002
LAT-003	Admin-share access across multiple hosts. SMB connections (445) from one host to three or more distinct internal hosts inside ten minutes.	threshold	medium	T1021.002, T1018
LAT-004	Workstation-to-workstation RDP. RDP (3389) or WinRM (5985/5986) initiated from a workstation to another workstation — jump-host bypass.	match	medium	T1021.001, T1021.006
LAT-005	Internal port scan. One host touching fifteen or more distinct internal destination ports/hosts within one minute.	threshold	medium	T1046, T1018
LAT-006	Pass-the-hash style NTLM logon. Network logon with NTLM from a workstation using a privileged account outside working hours pattern.	match	high	T1550.002

Table A.5 — Collection, command-and-control and exfiltration pack (collection_exfiltration.yaml, 16 rules).

ID	Rule	Kind	Severity	ATT&CK
COLL-001	Data staged into an archive. A large archive (zip/7z/rar/tar.gz) created in Temp / Public / Downloads / a hidden folder.	match	medium	T1560.001, T1074.001
COLL-002	Archive utility invoked with password / recursive flags. 7z / rar / tar compressing directories with a password — attacker staging.	match	medium	T1560.001

COLL-003	Mass access to sensitive file shares. Two hundred or more file reads from share / document paths by one user in five minutes.	threshold	medium	T1039, T1005
C2-001	Beaconing to an external host. Twenty-five or more connections from one host to the same external IP in thirty minutes (regular check-ins).	threshold	high	T1071.001, T1573
C2-002	Outbound connection on a non-standard port. Connection to a public IP on a port commonly used by C2 frameworks (4444, 1337, 6666, 8081, 9001, 50050).	match	high	T1571, T1071
C2-003	Script interpreter making outbound connection. PowerShell / wscript / mshta / rundll32 / regsvr32 opened a network connection to a public IP.	match	high	T1059.001, T1071.001
EXFIL-001	Large outbound transfer to an unsanctioned destination. A single connection sending more than 50 MB to a public destination that is NOT a sanctioned corporate cloud (Microsoft 365 / OneDrive / Google Workspace / approved backup).	match	high	T1048, T1041
EXFIL-002	Upload to personal cloud / file-sharing service. Outbound data to consumer file-sharing or paste domains.	match	high	T1567.002
EXFIL-003	Cumulative outbound volume to one external IP. More than 200 MB sent to a single external destination within one hour across connections.	threshold	high	T1041, T1030
FILE-001	Mass file modification (ransomware behaviour). One process modifying 150+ files on a host inside sixty seconds.	threshold	critical	T1486
FILE-002	Ransom note dropped. File named like a ransom note (README_TO_DECRYPT, HOW_TO_RECOVER, RESTORE-FILES.txt).	match	critical	T1486
FILE-003	Sensitive credential file accessed. Reads of password stores, browser credential DBs, cloud CLI tokens or SSH keys.	match	high	T1552.001, T1555.003, T1552.004
FILE-004	Executable written by an Office application or browser to a startup-ish location. winword/excel/outlook/browser writing .exe/.dll/.ps1/.vbs/.hta to disk.	match	high	T1566.001, T1204.002
DNS-001	DNS query to known dynamic-DNS / tunnelling provider. Lookups for free dynamic-DNS or DNS-tunnel providers frequently used for C2.	match	medium	T1071.004, T1568
DNS-002	Excessive NXDOMAIN responses. Fifty or more failed DNS resolutions from one host in two minutes — DGA behaviour.	threshold	medium	T1568.002
DNS-003	TXT-record query storm. Twenty or more TXT queries from one host in five minutes — DNS tunnelling channel.	threshold	medium	T1071.004, T1048.003

Table A.6 — Behavioural chains pack (behavioral_chains.yaml, 7 rules).

ID	Rule	Kind	Severity	ATT&CK
BEHAV-001	Office → shell → external network. An Office application spawned a script host which then connected to the internet within ten minutes.	sequence	critical	T1566.001, T1059.001, T1071.001

BEHAV-002	Encoded PowerShell → C2 → staged archive → exfiltration. The full credential-compromise-to-exfiltration chain: encoded PowerShell, outbound connection from the interpreter, an archive staged on disk, then a large outbound transfer.	sequence	critical	T1059.001, T1071.001, T1560.001, T1041
BEHAV-003	Login → privilege escalation → persistence. A successful login followed by privilege escalation and a new service / scheduled task on the same host.	sequence	critical	T1078, T1068, T1543.003
BEHAV-004	Archive staged then transmitted externally. A large archive was created and, within thirty minutes, a comparably large outbound transfer left the host.	sequence	high	T1560.001, T1048
BEHAV-005	Discovery → credential dumping → lateral movement. Reconnaissance, then LSASS/SAM access, then remote execution — the hands-on-keyboard intrusion pattern.	sequence	critical	T1087, T1003.001, T1021.002
BEHAV-006	Shadow copies deleted → mass encryption. Recovery inhibited immediately before mass file modification — ransomware detonation.	sequence	critical	T1490, T1486
BEHAV-007	Anomalous login followed by script execution. A login with a non-approved geolocation followed by PowerShell/cmd within thirty minutes on the same host.	sequence	high	T1078, T1059

APPENDIX B

REST API reference

Base URL `/api`; interactive OpenAPI documentation is served at `/docs` on any deployment. Humans authenticate at `POST /api/auth/login` and send the returned JWT as a bearer token; collectors authenticate with the `x-api-key` header. Roles are ordered ingestor (5) < viewer (10) < analyst (20) < admin (30); a route's role is the minimum required, and calls below it receive 403.

Table B.1 — The 24 endpoints by area (from `backend/aegis/api/routers.py` and `main.py`).

Method	Path	Role	Purpose
POST	<code>/auth/login</code>	—	Exchange credentials for a JWT
GET	<code>/auth/me</code>	any	Current identity: username, role, tenant
POST	<code>/ingest</code>	API key	Normalise and ingest a batch of raw collector records (≤5,000 events, ≤2 MB)
GET	<code>/overview</code>	viewer	Threat level, counts, distributions, tactic coverage, top incidents, pipeline stats
GET	<code>/incidents</code>	viewer	Filtered incident summaries (severity, status, limit)
GET	<code>/incidents/{id}</code>	viewer	Full incident with events and critical path
GET	<code>/incidents/{id}/graph</code>	viewer	Attack graph nodes/edges with layered layout
POST	<code>/incidents/{id}/status</code>	analyst	Update status: open, investigating, contained, resolved, false positive
POST	<code>/incidents/{id}/investigate</code>	analyst	Run the investigation pipeline; returns narrative, timeline, findings, actions, grounding
POST	<code>/incidents/{id}/copilot</code>	analyst	Ask a question; answer returns with cited evidence and grounding block
GET	<code>/graph/entity</code>	viewer	Neighbourhood of an entity in the knowledge graph (query, depth)
GET	<code>/graph/search</code>	viewer	Fuzzy entity search
GET	<code>/graph/threat-map</code>	viewer	External infrastructure across incidents with TI context
GET	<code>/threat-intel/stats</code>	viewer	Loaded indicator counts by type and feed
GET	<code>/threat-intel/lookup</code>	viewer	Look up an IP, domain, hash or URL
GET	<code>/rules</code>	viewer	All loaded rules with live fire counts
GET	<code>/mitre/coverage</code>	viewer	Catalogue technique coverage as a tactics tree
GET	<code>/mitre/observed</code>	viewer	Techniques observed across current incidents

POST	/simulate	admin	Inject an attack scenario (A–H) into the live platform
GET	/simulate/scenarios	viewer	List available scenarios
GET	/audit	admin	Recent audit entries with hash-chain verification
GET	/healthz	—	Liveness, event/incident counts, LLM availability
GET	/metrics	—	Prometheus exposition
GET	/	—	Service banner

Every response sets the security headers of §9.3 and an `X-Response-Time-ms` header; unhandled errors return a generic 500. Full request/response JSON shapes are specified in the repository's `docs/API_CONTRACT.md`.

APPENDIX C

The canonical event schema

The complete field set of `SecurityEvent` (`backend/aegis/schemas/events.py`). Configuration: `str_max_length=8192`, `extra="forbid"`; timestamps are coerced to timezone-aware UTC; `tags` is capped at 32 entries.

Table C.1 — `SecurityEvent` fields.

Field	Type	Meaning
<code>event_id</code>	<code>str</code>	Generated <code>evt_</code> + 12 hex chars; the identifier cited by every downstream consumer
<code>tenant_id</code>	<code>str</code>	Tenant scope; stamped at ingestion
<code>timestamp</code>	<code>datetime (UTC)</code>	When the event occurred
<code>source</code>	<code>enum (9 values)</code>	<code>windows</code> , <code>linux</code> , <code>network</code> , <code>dns</code> , <code>cloud</code> , <code>application</code> , <code>edr</code> , <code>identity</code> , <code>simulator</code>
<code>event_type</code>	<code>enum (20 values)</code>	Canonical type (Table 5.1)
<code>action</code>	<code>str</code>	Verb-like action: <code>login_success</code> , <code>connect</code> , <code>create</code> , ...
<code>outcome</code>	<code>str?</code>	<code>success</code> , <code>failure</code> , <code>blocked</code> , <code>unknown</code>
<code>host</code> , <code>user</code> , <code>session_id</code>	<code>str?</code>	Core identity entities (correlation keys, §5.3)
<code>process_name</code> , <code>process_id</code>	<code>str?</code> , <code>int?</code>	Executing process
<code>parent_process_name</code> , <code>parent_process_id</code>	<code>str?</code> , <code>int?</code>	Parent process (process-tree reasoning)
<code>command_line</code>	<code>str?</code>	Full command line — untrusted attacker-controllable text
<code>file_path</code> , <code>file_hash</code> , <code>file_size</code>	<code>str?</code> , <code>str?</code> , <code>int?</code>	File activity details
<code>src_ip</code> , <code>src_port</code>	<code>str?</code> , <code>int?</code>	Connection source
<code>dst_ip</code> , <code>dst_port</code>	<code>str?</code> , <code>int?</code>	Connection destination (public destinations become weak correlation keys)
<code>protocol</code>	<code>str?</code>	Transport/application protocol
<code>domain</code> , <code>url</code>	<code>str?</code>	DNS/HTTP context; both feed IOC extraction
<code>bytes_in</code> , <code>bytes_out</code>	<code>int?</code>	Transfer volumes; <code>bytes_out</code> feeds the egress baseline
<code>geo_country</code>	<code>str?</code>	Geolocation for authentication events

<code>privilege,</code> <code>target_user</code>	<code>str?</code>	Privilege-change and account-management context
<code>service_name</code>	<code>str?</code>	Service lifecycle events
<code>message</code>	<code>str?</code>	Free-text log message — untrusted
<code>severity_hint</code>	<code>enum</code>	Source-supplied severity (info by default); never overrides computed severity
<code>tags</code>	<code>list[str] ≤ 32</code>	Labels from collectors or the normaliser
<code>raw</code>	<code>dict</code>	Original record, kept verbatim for forensics; ignored by every engine

Helper methods: `entity_keys()` (correlation identity), `fingerprint()` (SHA-256-based dedup key over the identity-bearing fields), `short()` (single-line rendering used by reports, prompts and the console's evidence panels) — each discussed in §5.3.

APPENDIX D

Reproduction guide

Every claim in this thesis can be checked from a clean machine with Docker (or Python 3.11+ and Node 20+). Commands are as in the repository's README and runbook at the benchmarked revision.

D.1 Run the platform

```
git clone https://github.com/raunitgrey7/aegis.git && cd aegis
docker compose up --build          # API :8000 · console :3000
# optional profiles – local model + dashboards:
docker compose --profile ai --profile observability up --build
docker compose exec ollama ollama pull llama3.1:8b # once, for the AI narrative
```

Open `http://localhost:3000` and sign in with `analyst / analyst` (see §12.1 for the role set; change all demo credentials for any non-demo use). Without Docker: `pip install -e backend -e simulator`, then `uvicorn aegis.main:app --reload` and `npm install && npm run dev` in `frontend/`.

D.2 Reproduce the benchmark of Chapter 11

```
python -m aegis_sim.evaluation --attacks 100 --benign 100 --seed 1337 --out
evaluation/results
```

This regenerates `results.json`, `results.md` and `example_incidents.json`. The run is deterministic: the same seed yields the same numbers on any machine. Any other seed yields a comparable run with different draws. The language model is not involved regardless of what is installed.

D.3 Drive a live attack demonstration

```
python scripts/seed_demo.py --scenario C          # malicious document → C2
curl -s localhost:8000/api/incidents -H "Authorization: Bearer $TOKEN" | jq
'.incidents[0]'
curl -s -X POST localhost:8000/api/incidents/SEC-0001/investigate \
-H "Authorization: Bearer $TOKEN" | jq '.grounding'
```

D.4 Run the tests and build the deck

```
make test      # 57 backend tests
make lint
python pitch/build_deck.py  # pitch deck generated from real results.json
```

APPENDIX E

A worked incident, end to end

The evaluation harness commits one fully worked incident per scenario (`evaluation/results/example_incidents.json`); these seed the demo console and this appendix. What follows is the scenario-C example exactly as the pipeline emitted it — a malicious-document campaign against the workstation of a sales user, erin — traced through every mechanism in Chapters 6–8. All identifiers, scores and graph contents below are from the committed artefact, not reconstructed.

E.1 What the engines saw

The incident — titled *Command-and-control activity from WS-005*, severity critical, risk 100.0, confidence 0.93 — assembled eight detections from all three engine families across five underlying events (Table E.1). Reading the table top to bottom is reading the campaign: a login from a non-approved country; Word spawning a shell; an encoded PowerShell command; the script interpreter reaching out; and both the destination address and the contacted domain matching curated Cobalt Strike indicators. Two behavioural chains — Office→shell→external network, and anomalous-login-then-script — stitched the stages together, each citing two events.

Table E.1 — The eight detections behind the scenario-C example incident, as committed.

Rule	Title	Severity	Score	Conf.Phase
AUTH-004	Successful login from a non-approved country	medium	35.0	0.70initial access
BEHAV-007	Anomalous login followed by script execution	high	55.0	0.85initial access
EXEC-002	Office application spawned a shell	high	55.0	0.92execution
EXEC-001	Encoded PowerShell command	high	55.0	0.93execution
BEHAV-001	Office → shell → external network	critical	65.0	0.93execution
C2-003	Script interpreter making outbound connection	high	50.0	0.90command & control
TI-IP	Known malicious IP: Cobalt Strike C2	critical	72.75	0.90command & control
TI-DOMAIN	Known malicious domain: Cobalt Strike malleable C2	critical	72.75	0.90command & control

E.2 How the score assembled

The committed `score_breakdown` itemises the §7.5 formula's work. The confidence-weighted noisy-OR over the eight detections reaches 99.9 — eight independent lines of evidence leave essentially no probability of innocence to multiply away. Three distinct phases (initial access 4, execution 4, command-and-control 6) yield a kill-chain bonus of $(4+4+6) \times 0.6 = 8.4$. Two threat-intelligence hits earn the capped bonus of 12.0. WS-005 is an ordinary workstation and erin an ordinary account, so asset criticality adds nothing; one host, so breadth adds nothing. Raw total 120.3, capped to 100.0 — and every number in this paragraph is displayed, verbatim, in the console's score panel.

Table E.2 — The committed `score_breakdown` for the example incident.

Term	Value
Detections (confidence-weighted noisy-OR)	99.9
Kill-chain bonus (3 phases: $4 + 4 + 6, \times 0.6$)	8.4
Threat-intelligence bonus (2 hits, capped)	12.0
Asset-criticality bonus	0.0
Breadth bonus	0.0
Raw total	120.3
Risk (capped)	100.0

E.3 The graph and the techniques

The extracted attack graph holds ten nodes and ten edges: the user (erin, node risk 65), her source address, the workstation (WS-005, 72.8), three processes (explorer.exe → winword.exe → powershell.exe — the parent-child chain that makes EXEC-002 undeniable), the external address 45.155.205.12 and domain update-cdn-service.net (both 72.8), and the two threat-intelligence indicator nodes (risk 90) that annotate them. Nine ATT&CK techniques aggregate from the detections — among them T1566.001 (spear-phishing attachment), T1059.001 (PowerShell), T1027.010 (command obfuscation) and T1071.001 (web-protocol C2) — against the scenario's four-technique ground truth: a concrete instance of the recall-over-precision trade measured in §11.3. Everything an analyst sees in Figures 12.3 and 12.4 for the demo's equivalent incident is this structure, rendered.

APPENDIX F

Configuration reference

The platform's entire configuration surface is one typed settings object (`backend/aegis/config.py`), populated from environment variables with the `AEGIS_` prefix (or a local `.env`). Table F.1 lists the operationally significant settings with their defaults; the two marked [△](#) ship with demo values that must be changed for any non-demo deployment (§9.6).

Table F.1 — Settings and defaults (environment variable = `AEGIS_` + upper-cased name).

Setting	Default	Governs
<code>env / log_level</code>	<code>development / INFO</code>	Environment name; logging verbosity
<code>api_host / api_port</code>	<code>0.0.0.0 / 8000</code>	API bind address
<code>database_url</code>	SQLite file	Persistence; point at PostgreSQL for production
<code>redis_url</code>	None	Event bus; unset selects the in-memory bus (§4.4)
<code>event_stream / consumer_group</code>	<code>aegis:events / aegis-detectors</code>	Stream name and consumer group
<code>jwt_secret</code> △	placeholder	JWT signing key (HS256)
<code>jwt_expiry_minutes</code>	480	Token lifetime
<code>ingest_api_key</code> △	<code>aegis-dev-ingest-key</code>	Collector credential for <code>/ingest</code>
<code>admin_username / admin_password</code> △	<code>admin / admin</code>	Bootstrap account (demo only)
<code>max_request_bytes</code>	2097152	Request-size cap enforced in middleware (§9.2)
<code>max_events_per_batch</code>	5000	Ingest batch cap
<code>rate_limit_per_minute</code>	600	Token-bucket rate limiter (§9.3)
<code>default_tenant</code>	default	Tenant stamped at ingestion (§4.5)
<code>correlation_window_seconds</code>	3600	Clustering window (§7.1)
<code>incident_min_score</code>	40.0	Admission floor (§7.2)
<code>ollama_url / ollama_model</code>	<code>localhost:11434 / llama3.1:8b</code>	Local model endpoint and name
<code>llm_enabled / llm_timeout_seconds</code>	<code>true / 90</code>	Master switch and generation timeout for the AI layer

<code>embedding_model</code>	<code>nomic-embed-text</code>	Local embedding model (optional pgvector features)
<code>otel_enabled /</code> <code>otel_endpoint</code>	<code>false /</code> <code>localhost:4317</code>	OpenTelemetry export

The defaults encode the degradation story: with nothing set at all, the platform boots on SQLite and the in-memory bus with the model enabled-but-absent — the exact configuration the public demo runs, and the baseline configuration every claim in this thesis holds under.

APPENDIX G

Glossary

Table G.1 — Terms as used in this thesis.

Term	Meaning here
Admission policy	The checks deciding whether a correlated cluster becomes an incident (§7.2)
Attack graph	The graph of entities and relationships induced by an incident's evidence events, laid out in provenance layers (§7.4)
Baseline	A per-entity statistical summary of benign history that an anomaly detector scores against (§6.5)
Beaconing	Periodic outbound contact from an implant to its command infrastructure
Cooldown	Per-rule, per-group suppression window preventing duplicate detections for one behaviour burst (§6.7)
Correlation window	The time span (default 1 h) within which detections sharing an entity are clustered (§7.1)
Detection	A structured finding from one engine, carrying score, confidence, phase, techniques and evidence identifiers (§6.1)
Evidence coverage	Fraction of a scenario's ground-truth events attached to its incident (Table 10.2)
Fingerprint	The stable hash making event re-ingestion idempotent (§5.3)
Grounding	Validation of AI output against the incident's real evidence identifiers; fabrication reverts the output (§8.3)
Hard negative	Benign activity engineered to resemble an attack, used to price the false-positive rate (§10.3)
Honeytoken	A decoy credential or account whose only legitimate use is none; any use is a detection
Incident	An admitted cluster of correlated detections with graph, phases, techniques, risk and status (§7.2)
Indicator (IOC)	An address, domain, URL, hash or range attested malicious by a feed (§6.6)
Kill chain	The phase structure of an intrusion campaign; Aegis annotates thirteen phases from reconnaissance to impact (§2.3)
Lateral movement	An intruder extending control from one host to others inside the estate
Living off the land	Attacker use of legitimate administrative tooling to evade signature detection
Noisy-OR	The probabilistic combination under which independent evidence compounds without exceeding certainty (§7.5)
Normalisation	Mapping vendor-shaped records onto the canonical event schema (§5.4)

Prompt injection	Steering a language model through attacker-controlled text in its input; indirect when the text arrives as data (§9.2)
Robust z-score	Deviation measured against median/MAD rather than mean/ σ , resistant to outlier contamination (§6.5)
Sequence rule	A rule matching ordered multi-event behaviour within a window (§6.2)
Tenant	The isolation scope stamped on every event, detection and incident (§4.5)
Threat intelligence	External knowledge of adversary infrastructure, consumed here from public feeds only (§6.6)

APPENDIX H

The committed benchmark report

For completeness, the machine-generated Markdown report of the committed benchmark run is reproduced below exactly as it stands in the repository (`evaluation/results/results.md`). It is the artefact from which every number in Chapter 11 and the project README is drawn, and the artefact that `python -m aegis_sim.evaluation --attacks 100 --benign 100 --seed 1337` regenerates byte-for-byte on any machine.

```
# Aegis Evaluation Report

_Generated 2026-08-30T14:38:53.881941+00:00 · seed 1337 · deterministic & reproducible_

Ran **100 attack** and **100 benign** scenarios against a synthetic 60-user enterprise after a
**23,047-event** benign training baseline (**30,496** events total).

## Headline metrics

| Metric | Result |
|-----|-----:|
| Detection Rate (recall) | **100.0%** |
| False Positive Rate | **2.0%** |
| Precision | 98.0% |
| F1 score | 99.0% |
| Attack-chain reconstruction | 89.5% |
| MITRE technique recall | 86.7% |
| MITRE technique precision | 62.2% |
| IOC correlation accuracy | 82.9% |
| Evidence coverage | 73.4% |
| Mean detection latency | 766.84 ms |
| p95 detection latency | 2269.16 ms |
| Throughput | 199 events/s |

Confusion matrix – TP 100, FP 2, TN 98, FN 0.

## Per-scenario breakdown

| ID | Scenario | Runs | Det. rate | Chain recon. | Technique recall | Evidence cov. | Mean risk |
|----|-----|-----:|-----:|-----:|-----:|-----:|-----:|
| A | Brute-force authentication | 13 | 100.0% | 100.0% | 100.0% | 55.5% | 93.5 |
| B | Suspicious off-hours login | 13 | 100.0% | 50.0% | 66.7% | 100.0% | 66.6 |
| C | Malicious document execution | 13 | 100.0% | 100.0% | 100.0% | 75.0% | 100.0 |
| D | Privilege escalation & backdoor account | 13 | 100.0% | 100.0% | 100.0% | 80.0% | 100.0 |
| E | Hands-on-keyboard lateral movement | 12 | 100.0% | 66.7% | 75.0% | 29.9% | 100.0 |
| F | Ransomware detonation | 12 | 100.0% | 100.0% | 100.0% | 71.7% | 100.0 |
| G | DNS tunnelling / exfiltration channel | 12 | 100.0% | 100.0% | 100.0% | 98.9% | 100.0 |
| H | Data collection & exfiltration | 12 | 100.0% | 100.0% | 50.0% | 75.0% | 100.0 |

_Detection is fully deterministic; the LLM is not involved in any number above._
```

Listing H.1 — `evaluation/results/results.md`, verbatim.

— end of document —